

1 Appendix A: Constructible Persistence and Barcode-Level Threshold Collapse

A.1. Scope and relation to Chapter 2

This appendix supplies the persistence-theoretic foundation for Chapter 2 after the v18.1 Core repair. Its purpose is to justify, in a self-contained way, the repaired collapse apparatus used in the Core:

$$\text{Pers}_k^{\text{cons}}, \quad \mathcal{B}(M), \quad T_\tau^{\text{bar}}, \quad \text{Eval}_{i,W,\tau}.$$

The central correction is the following:

For a positive threshold $\tau > 0$, the class of constructible persistence modules built entirely from bars of length at most τ is not a Serre subcategory in general. Therefore positive-threshold bar deletion is not asserted, in the general Core regime, to be an exact Serre localization or an abelian reflector.

Accordingly, this appendix supports the following repaired Core assertions:

- (i) constructible one-parameter persistence over a field provides the barcode regime used by the Core;
- (ii) constructible persistence modules admit interval decompositions;
- (iii) positive-threshold short-bar objects are not Serre in general;
- (iv) T_τ^{bar} is a barcode-level threshold-deletion operation;
- (v) T_τ^{bar} is endpoint-independent, idempotent, threshold-monotone, and barcode-stable;
- (vi) window restriction can shorten bars and therefore does not commute with threshold deletion in general;
- (vii) the Core uses the fixed windowed evaluation order

$$\text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(\mathbf{W}_W \mathbf{P}_i(F));$$

- (viii) stronger functorial or exact statements require separately declared τ -split or exact-admissible regimes.

Remark 1.1 (Appendix A does not enlarge the Core). This appendix does not introduce problem-specific claims, Search-layer procedures, AI-agent semantics, AWFS structures, Mirror/Transfer 2-cell control, or arithmetic/PDE interfaces. It supplies only the persistence-theoretic background required by the repaired Chapter 2.

A.2. One-parameter persistence modules

Fix a field k . Let

$$(\mathbb{R}, \leq)$$

be the usual ordered real line.

Definition 1.2 (Persistence module). A *one-parameter persistence module over k* is a functor

$$M : (\mathbb{R}, \leq) \longrightarrow \text{Vect}_k.$$

Thus, for each $t \in \mathbb{R}$, one has a vector space $M(t)$, and for each $s \leq t$, a structure map

$$M(s \leq t) : M(s) \longrightarrow M(t),$$

satisfying the usual functorial identities.

Definition 1.3 (Category of persistence modules). We denote by

$$\text{Pers}_k := [(\mathbb{R}, \leq), \text{Vect}_k]$$

the category of one-parameter persistence modules over k , with natural transformations as morphisms.

Remark 1.4 (Pointwise abelian structure). Since Vect_k is abelian and limits and colimits in a functor category are computed pointwise, Pers_k is abelian. For a morphism $f : M \rightarrow N$, its kernel and cokernel are given by

$$(\ker f)(t) = \ker(f_t), \quad (\text{coker } f)(t) = \text{coker}(f_t),$$

with structure maps induced pointwise.

A.3. Constructible persistence modules

Definition 1.5 (Critical set). Let $M \in \text{Pers}_k$. A finite subset

$$S = \{s_1 < \dots < s_m\} \subset \mathbb{R}$$

is called a *critical set* for M on an interval $I \subset \mathbb{R}$ if, on every connected component of

$$I \setminus S,$$

all structure maps of M are isomorphisms.

Definition 1.6 (Constructible persistence module). A persistence module $M \in \text{Pers}_k$ is called *constructible* if:

- (i) $M(t)$ is finite-dimensional over k for all $t \in \mathbb{R}$;
- (ii) on every bounded interval $I \subset \mathbb{R}$, there exists a finite critical set for $M|_I$;
- (iii) M has no wild accumulation of bar endpoints inside bounded intervals.

We denote the full subcategory of constructible persistence modules by

$$\text{Pers}_k^{\text{cons}} \subset \text{Pers}_k.$$

Remark 1.7 (Local finiteness). The constructibility condition is local on bounded intervals. It allows infinitely many bars globally, provided that only finitely many changes occur in each bounded region. For the AK Core, all monitored evaluations are performed on declared windows or locally finite regimes.

Proposition 1.8 (Constructible kernels and cokernels). *Let*

$$f : M \rightarrow N$$

be a morphism in $\text{Pers}_k^{\text{cons}}$. Then the pointwise kernel and cokernel of f , computed in Pers_k , are again constructible.

Proof. Pointwise kernels and cokernels exist because Vect_k is abelian. Since $M(t)$ and $N(t)$ are finite-dimensional for all t , the vector spaces

$$(\ker f)(t), \quad (\text{coker } f)(t)$$

are finite-dimensional. On a bounded interval, take a common finite refinement of critical sets for M , N , and the morphism f . Away from this finite refinement, the structure maps of M and N are isomorphisms and the naturality squares commute. The induced structure maps on kernels and cokernels are therefore also isomorphisms away from the same finite set. Thus the kernel and cokernel are constructible. $\square$

Remark 1.9 (Abelian working regime). The preceding proposition gives the abelian working regime needed for ordinary persistence morphisms. The v18.1 repair does not use this abelian structure to claim that positive-threshold short-bar deletion is an exact Serre localization. The abelianity of $\text{Pers}_k^{\text{cons}}$ and the exactness of T_τ^{bar} are distinct issues.

A.4. Interval modules and barcode decomposition

Definition 1.10 (Interval module). For an interval $I \subseteq \mathbb{R}$, the *interval module* k_I is the persistence module defined by

$$k_I(t) = \begin{cases} k, & t \in I, \\ 0, & t \notin I, \end{cases}$$

with structure maps equal to the identity on k whenever both source and target lie in I , and zero otherwise.

Definition 1.11 (Bar and barcode). An interval summand k_I is called a *bar*. A barcode of $M \in \text{Pers}_k^{\text{cons}}$ is a locally finite multiset of intervals

$$\mathcal{B}(M) = \{I_\lambda\}_{\lambda \in \Lambda}$$

representing a decomposition

$$M \cong \bigoplus_{\lambda \in \Lambda} k_{I_\lambda}.$$

Theorem 1.12 (Interval decomposition). *Every object $M \in \text{Pers}_k^{\text{cons}}$ admits an interval decomposition*

$$M \cong \bigoplus_{\lambda \in \Lambda} k_{I_\lambda},$$

where the barcode is locally finite on bounded intervals. Moreover, the barcode is unique up to permutation of interval summands.

Proof sketch. This is the standard interval decomposition theorem for pointwise finite-dimensional constructible one-parameter persistence modules over a field. On bounded intervals, constructibility reduces the module to a finite representation of a linearly ordered finite poset. Such representations decompose into interval representations. Compatibility over bounded intervals and local finiteness yield the global locally finite barcode. Uniqueness follows from the Krull–Schmidt property in the constructible interval-decomposable regime. $\square$

Remark 1.13 (Barcode normal form as the Core input). The repaired AK Core uses barcode normal forms as the input for threshold deletion. This is weaker than asserting a global exact localization, but it is sufficient for windowed persistence evaluation, metric stability, and gate-readable collapse profiles.

A.5. Length of bars and endpoint conventions

Definition 1.14 (Length of a finite bar). Let $I \subset \mathbb{R}$ be a bounded interval with endpoints $a < b$. The *length* of I is

$$\ell(I) := b - a.$$

This length is independent of whether endpoints are open, closed, or half-open.

Definition 1.15 (Infinite bar). A bar I is called *infinite* if it is unbounded above. For thresholding purposes, infinite bars are treated as having length

$$\ell(I) = +\infty.$$

Remark 1.16 (Endpoint independence). Since bar length depends only on endpoint values, the threshold test

$$\ell(I) \leq \tau$$

is invariant under harmless endpoint bookkeeping. Thus barcode-level threshold deletion is endpoint-independent.

A.6. Failure of the positive-threshold Serre property

Fix $\tau > 0$.

Definition 1.17 (Short-bar class). Let

$$\mathcal{S}_\tau$$

denote the class of constructible persistence modules whose barcode summands are all finite bars of length at most τ . Equivalently,

$$M \in \mathcal{S}_\tau$$

if and only if every bar $I \in \mathcal{B}(M)$ is finite and satisfies

$$\ell(I) \leq \tau.$$

Proposition 1.18 (Positive-threshold short bars are not Serre in general). *For $\tau > 0$, the class $\mathcal{S}_\tau$ is not extension-closed in general. Consequently, it is not a Serre subcategory of $\text{Pers}_k^{\text{cons}}$ in general.*

Proof. It suffices to exhibit a short exact sequence in which the subobject and quotient belong to $\mathcal{S}_\tau$, while the middle object does not.

For $\tau = 1$, consider

$$0 \longrightarrow k_{[1,2)} \longrightarrow k_{[0,2)} \longrightarrow k_{[0,1)} \longrightarrow 0.$$

The subobject $k_{[1,2)}$ has length 1, and the quotient $k_{[0,1)}$ has length 1. However, the middle object $k_{[0,2)}$ has length 2, hence does not belong to $\mathcal{S}_1$.

For a general $\tau > 0$, rescale the intervals:

$$0 \longrightarrow k_{[\tau, 2\tau)} \longrightarrow k_{[0, 2\tau)} \longrightarrow k_{[0, \tau)} \longrightarrow 0.$$

Again, the subobject and quotient have length τ , while the middle object has length 2τ . Thus $\mathcal{S}_\tau$ is not extension-closed. $\square$

Corollary 1.19 (No global Serre localization). *For $\tau > 0$, the Core does not assert that deletion of bars of length at most τ arises from a Serre localization of $\text{Pers}_k^{\text{cons}}$ by $\mathcal{S}_\tau$.*

Corollary 1.20 (No global exact reflector). *For $\tau > 0$, positive-threshold bar deletion is not asserted in the general Core to be a global exact reflector on all of $\text{Pers}_k^{\text{cons}}$.*

Remark 1.21 (Why this repair is necessary). If S_τ were incorrectly treated as Serre, then the quotient construction

$$M \mapsto M/S_\tau$$

would appear to provide exactness, functoriality, and universal properties that are not justified in general. The repaired theory avoids this claim and uses threshold deletion only at the barcode-profile level unless additional exact-admissible hypotheses are declared.

A.7. Barcode-level threshold deletion

Definition 1.22 (Barcode-level threshold deletion). Let $M \in \text{Pers}_k^{\text{cons}}$ have barcode

$$\mathcal{B}(M) = \{I_\lambda\}_{\lambda \in \Lambda}.$$

For $\tau > 0$, define

$$T_\tau^{\text{bar}}(M)$$

to be the barcode profile obtained by retaining exactly those bars satisfying

$$\ell(I_\lambda) > \tau \quad \text{or} \quad \ell(I_\lambda) = +\infty.$$

Equivalently,

$$T_\tau^{\text{bar}}(M) := \bigoplus_{\lambda: \ell(I_\lambda) > \tau \text{ or } \ell(I_\lambda) = +\infty} k_{I_\lambda}$$

as a barcode normal form.

Remark 1.23 (Operation on barcode normal forms). In the general Core, T_τ^{bar} is an operation on barcode normal forms and hence on isomorphism classes of constructible persistence modules. It is not asserted, without additional hypotheses, to be a strict exact endofunctor on every morphism of $\text{Pers}_k^{\text{cons}}$.

Proposition 1.24 (Endpoint independence of T_τ^{bar}). *The operation T_τ^{bar} depends only on bar lengths. Therefore:*

- (i) *finite bars of length $\leq \tau$ are deleted;*
- (ii) *finite bars of length $> \tau$ survive;*
- (iii) *infinite bars survive;*
- (iv) *endpoint-open/closed conventions do not affect the result.*

Proof. The defining rule for T_τ^{bar} uses only the numerical value $\ell(I)$. Endpoint inclusion does not change $\ell(I)$, and infinite bars have length $+\infty$. $\square$

Proposition 1.25 (Idempotence). *For every $\tau > 0$,*

$$T_\tau^{\text{bar}} T_\tau^{\text{bar}}(M) \cong T_\tau^{\text{bar}}(M)$$

at the barcode-profile level.

Proof. After applying T_τ^{bar} once, no finite bar of length at most τ remains. Applying the same deletion rule again removes nothing further. $\square$

Proposition 1.26 (Threshold composition). *For $\tau, \sigma > 0$,*

$$T_\sigma^{\text{bar}} T_\tau^{\text{bar}}(M) \cong T_{\max\{\tau, \sigma\}}^{\text{bar}}(M)$$

at the barcode-profile level.

Proof. The first deletion removes finite bars of length at most τ . The second deletion removes finite bars of length at most σ among those remaining. The total effect is deletion of all finite bars of length at most

$$\max\{\tau, \sigma\}.$$

□

Definition 1.27 (Threshold comparison record). If

$$0 < \tau \leq \sigma,$$

then σ is a stronger collapse threshold than τ . At the barcode-profile level, the bars retained by $T_\sigma^{\text{bar}}(M)$ form a subprofile of the bars retained by $T_\tau^{\text{bar}}(M)$. The Core records this relation as

$$T_\tau^{\text{bar}}(M) \rightsquigarrow T_\sigma^{\text{bar}}(M).$$

Remark 1.28 (Direction of threshold comparison). The comparison direction is:

$$0 < \tau \leq \sigma \implies T_\tau^{\text{bar}} \rightsquigarrow T_\sigma^{\text{bar}}.$$

This is a retained-profile comparison from weaker collapse to stronger collapse. It is not a global natural transformation unless a functorial or exact-admissible regime is declared.

A.8. Interleaving distance and barcode stability

Definition 1.29 (Interleaving distance). Let $M, N \in \text{Pers}_k^{\text{cons}}$. The *interleaving distance*

$$d_{\text{int}}(M, N)$$

is the infimum of $\varepsilon \geq 0$ such that M and N are ε -interleaved. In the constructible one-parameter regime over a field, this agrees with the bottleneck distance on barcodes.

Theorem 1.30 (Barcode-level non-expansiveness). *For $M, N \in \text{Pers}_k^{\text{cons}}$,*

$$d_{\text{int}}(T_\tau^{\text{bar}}(M), T_\tau^{\text{bar}}(N)) \leq d_{\text{int}}(M, N)$$

at the barcode metric level.

Proof sketch. By the barcode isometry theorem, d_{int} is computed by the bottleneck distance between barcodes. Apply an optimal or ε -optimal barcode matching between $\mathcal{B}(M)$ and $\mathcal{B}(N)$. Deleting bars of length at most τ from both sides restricts the matching to the retained bars and cannot force a larger matching cost. Therefore the retained barcode profiles are no farther apart than the original profiles. □

Remark 1.31 (What stability does and does not imply). The non-expansiveness of T_τ^{bar} is a metric statement. It does not imply exactness, Serre localization, or functoriality on all morphisms. The repaired Core uses stability for audit error control, not as a substitute for exactness.

A.9. Window restriction

Definition 1.32 (Window restriction). Let

$$W = [u, u')$$

be a right-open window. For $M \in \text{Pers}_k^{\text{cons}}$, define

$$W_W(M)$$

by restricting M to W , and, when a global-indexed persistence module is required, extending by zero outside W . At the barcode level, W_W clips each bar I to

$$I \cap W,$$

discarding empty intersections.

Proposition 1.33 (Window restriction is barcode-stable). *For all $M, N \in \text{Pers}_k^{\text{cons}}$,*

$$d_{\text{int}}(W_W(M), W_W(N)) \leq d_{\text{int}}(M, N)$$

at the barcode metric level.

Proof sketch. Clipping all bars to the same fixed right-open window cannot increase the endpoint displacement needed to match corresponding bars. Equivalently, a barcode matching for M and N restricts to a matching for the clipped barcodes with no larger cost. $\square$

Remark 1.34 (Exactness of restriction is not the issue). Window restriction itself can be treated pointwise and is well behaved in the constructible regime. The problem is not restriction alone, but its interaction with length-threshold deletion, because restriction can shorten bars.

A.10. Failure of unconditional crop–collapse commutation

Proposition 1.35 (Window restriction and threshold deletion do not commute in general). *In general,*

$$W_W T_\tau^{\text{bar}}(M) \not\cong T_\tau^{\text{bar}} W_W(M).$$

Proof. Let

$$M = k_{[0,10)}, \quad W = [0, 1), \quad \tau = 2.$$

Since the global bar $[0, 10)$ has length $10 > \tau$, it survives T_τ^{bar} . Therefore

$$W_W T_\tau^{\text{bar}}(M) \cong k_{[0,1)}.$$

On the other hand,

$$W_W(M) \cong k_{[0,1)},$$

whose length is $1 \leq 2$. Thus

$$T_\tau^{\text{bar}} W_W(M) = 0.$$

Hence the two operations do not commute in general. $\square$

Remark 1.36 (Core consequence). The Core must not use unconditional identities of the form

$$W_W T_\tau^{\text{bar}} \cong T_\tau^{\text{bar}} W_W.$$

Instead, it uses a fixed evaluation order, or else requires a separately declared window-adapted commutation condition.

A.11. Windowed evaluation order

Definition 1.37 (Windowed collapsed persistence evaluation). Let $F \in \text{FiltCh}(\cdot)k$, let i be a monitored homological degree, let W be a right-open window, and let $\tau > 0$. The *windowed collapsed persistence evaluation* is

$$\text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Remark 1.38 (Fixed order). The expression

$$\text{Eval}_{i,W,\tau}(F)$$

always means the ordered procedure

$$F \mapsto \mathbf{P}_i(F) \mapsto W_W \mathbf{P}_i(F) \mapsto T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Thus the Core first restricts to the declared window and only then applies threshold deletion within that window.

Remark 1.39 (Why the order is chosen). The window is part of the certified local claim. Therefore the Core evaluates the persistence content actually visible inside the declared window. If collapse were applied before windowing, a globally long bar could survive and then appear inside a short window even though its window-visible segment is shorter than the threshold. The fixed order prevents this ambiguity.

A.12. Conditional window-adapted commutation

Definition 1.40 (τ -window-adapted pair). A pair (M, W) , with $M \in \text{Pers}_k^{\text{cons}}$ and W a right-open window, is called *τ -window-adapted* if every finite bar of M that survives T_τ^{bar} either:

- (i) remains of clipped length $> \tau$ after restriction to W ; or
- (ii) is disjoint from W .

Proposition 1.41 (Conditional crop–collapse commutation). *If (M, W) is τ -window-adapted, then*

$$W_W T_\tau^{\text{bar}}(M) \cong T_\tau^{\text{bar}} W_W(M)$$

at the barcode-profile level.

Proof. Under the τ -window-adapted condition, no bar that survives globally is shortened by window restriction into the deleted length range $\leq \tau$. Thus the retained bar profiles agree whether threshold deletion is applied before or after window restriction. $\square$

Remark 1.42 (Default policy). The default AK Core policy does not assume τ -window-adaptedness. It uses

$$T_\tau^{\text{bar}} W_W$$

as the fixed order. The conditional commutation result may be used only when the τ -window-adapted condition is explicitly declared.

A.13. Exact-admissible regimes

The repaired Core does not assert global exactness for T_τ^{bar} . However, exact or functorial collapse may be recovered in restricted regimes.

Definition 1.43 (τ -split object). An object $M \in \text{Pers}_k^{\text{cons}}$ is called τ -split if it is equipped with a declared decomposition

$$M \cong M_{\leq \tau} \oplus M_{> \tau}$$

such that:

- (i) every finite bar of $M_{\leq \tau}$ has length at most τ ;
- (ii) every finite bar of $M_{> \tau}$ has length greater than τ ;
- (iii) all infinite bars lie in $M_{> \tau}$;
- (iv) the decomposition is part of the declared data.

Definition 1.44 (τ -split morphism). A morphism

$$f : M \rightarrow N$$

between τ -split objects is called τ -split if it respects the declared decompositions:

$$f(M_{\leq \tau}) \subseteq N_{\leq \tau}, \quad f(M_{> \tau}) \subseteq N_{> \tau}.$$

Definition 1.45 (τ -split regime). A τ -split regime is a declared working subcategory or workflow domain in which all objects and morphisms under consideration are τ -split.

Proposition 1.46 (Functorial threshold deletion in a τ -split regime). *Inside a declared τ -split regime, the assignment*

$$M \mapsto M_{> \tau}$$

is functorial and agrees with $T_\tau^{\text{bar}}(M)$ on barcode profiles.

Proof. If $f : M \rightarrow N$ is τ -split, then

$$f(M_{> \tau}) \subseteq N_{> \tau}.$$

Thus f restricts to a morphism

$$M_{> \tau} \rightarrow N_{> \tau}.$$

Therefore $M \mapsto M_{> \tau}$ is functorial inside the declared τ -split regime. By definition, $M_{> \tau}$ is exactly the retained barcode profile after threshold deletion. $\square$

Definition 1.47 (Exact-admissible collapse regime). An *exact-admissible collapse regime* is a declared τ -split regime in which the retained-long-bar assignment

$$M \mapsto M_{> \tau}$$

is additionally verified to preserve the exact sequences used in the relevant Core argument.

Remark 1.48 (Conditional exactness only). Exactness of positive-threshold collapse is not a global theorem of Appendix A. It is available only after an exact-admissible regime is declared and verified. A proof object using exactness must record the split policy, morphism compatibility, exact sequences, and supporting artifacts.

A.14. Filtered representatives

Definition 1.49 (Windowed collapsed profile). For $F \in \text{FiltCh}(\cdot)k$, monitored degree i , window W , and threshold τ , the windowed collapsed profile is

$$\mathcal{P}_{i,W,\tau}(F) := \text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Definition 1.50 (Admissible filtered representative). A filtered object

$$C_{\tau,W}F$$

is called an admissible filtered representative of the windowed collapsed profile of F if, for every monitored degree i ,

$$\mathbf{P}_i(C_{\tau,W}F) \cong \mathcal{P}_{i,W,\tau}(F)$$

at the persistence barcode level, under the declared equivalence policy.

Remark 1.51 (No global strict filtered collapse). Appendix A does not assert a global strict endofunctor

$$C_\tau : \text{FiltCh}(\cdot)k \rightarrow \text{FiltCh}(\cdot)k$$

realizing positive-threshold collapse everywhere. The default object is the windowed representative

$$C_{\tau,W}F,$$

whose existence and properties must be supplied by the relevant implementation or proof record.

A.15. Tower comparison artifacts

Definition 1.52 (Windowed collapsed tower profile). Let

$$F_0 \rightarrow F_1 \rightarrow F_2 \rightarrow \dots$$

be a directed tower in the declared regime. For a monitored degree i , window W , and threshold τ , the associated windowed collapsed tower profile is the sequence

$$\mathcal{P}_{i,W,\tau}(F_0), \mathcal{P}_{i,W,\tau}(F_1), \mathcal{P}_{i,W,\tau}(F_2), \dots$$

together with declared comparison data between these profiles.

Definition 1.53 (Tower comparison artifact). A tower comparison artifact for (i, W, τ) is declared data specifying a comparison

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty)$$

between the collapsed profile of the tower and the collapsed profile of its declared apex or limiting object.

Remark 1.54 (No automatic tower comparison). Since T_τ^{bar} is not a global exact functor in the general Core, the tower comparison map

$$\phi_{i,W,\tau}$$

is not automatic. It must be either canonical in an exact-admissible regime or explicitly declared as an artifact.

A.16. Foundation package

Theorem 1.55 (Appendix A repaired foundation package). *Within the constructible one-parameter persistence regime over a field, the repaired Appendix A supplies the following package.*

- (i) *Constructible persistence modules admit locally finite barcode decompositions.*
- (ii) *Bar length is endpoint-independent.*
- (iii) *Positive-threshold short-bar objects do not form a Serre subcategory in general.*
- (iv) *Therefore positive-threshold bar deletion is not asserted to be a global exact Serre localization or exact reflector.*
- (v) T_τ^{bar} *is a barcode-level threshold-deletion operation.*
- (vi) T_τ^{bar} *is endpoint-independent and idempotent.*
- (vii) *Threshold composition satisfies*

$$T_\sigma^{\text{bar}} T_\tau^{\text{bar}} \cong T_{\max\{\tau, \sigma\}}^{\text{bar}}$$

at the barcode-profile level.

- (viii) *If $0 < \tau \leq \sigma$, the threshold comparison is*

$$T_\tau^{\text{bar}} \rightsquigarrow T_\sigma^{\text{bar}}.$$

- (ix) T_τ^{bar} *is non-expansive at the barcode metric level.*
- (x) *Window restriction is non-expansive at the barcode metric level.*
- (xi) *Window restriction and threshold deletion do not commute in general.*
- (xii) *The Core uses the fixed evaluation order*

$$\text{Eval}_{i, W, \tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

- (xiii) *Conditional crop–collapse commutation is available under the τ -window-adapted condition.*
- (xiv) *Exactness and strict functoriality are available only in declared exact-admissible regimes.*
- (xv) *Filtered representatives and tower comparison maps must be declared as proof or implementation data.*

Proof. Items (i)–(ii) follow from constructible interval decomposition and the definition of bar length. Item (iii) is Proposition 1.18. Item (iv) follows from Corollaries 1.19 and 1.20. Items (v)–(viii) follow from the definition and elementary properties of T_τ^{bar} . Item (ix) is Theorem 1.30. Item (x) is Proposition 1.33. Item (xi) is Proposition 1.35. Item (xii) is Definition 1.37. Item (xiii) is Proposition 1.41. Item (xiv) follows from Definitions 1.45 and 1.47. Item (xv) follows from Definitions 1.50 and 1.53. $\square$

A.17. Explicit non-claims

The following are not asserted in Appendix A.

- (i) No claim is made that positive-threshold short-bar objects form a Serre subcategory in general.
- (ii) No claim is made that T_τ^{bar} is a global exact endofunctor on all of $\text{Pers}_k^{\text{cons}}$.
- (iii) No claim is made that T_τ^{bar} is a global Serre localization.
- (iv) No claim is made that T_τ^{bar} is a global abelian reflector.
- (v) No quotient formula
$$T_\tau(M) = M/E_\tau(M)$$
is asserted in the general Core.
- (vi) No unconditional crop–collapse commutation is asserted.
- (vii) No global strict filtered lift C_τ is asserted.
- (viii) No tower comparison map after threshold deletion is automatic.
- (ix) No Ext-clause is supplied here; Ext usage requires the eligible realization discipline of the relevant later chapter or appendix.
- (x) No AWFS structure, Mirror/Transfer calculus, Search-layer potential, AI ranking, arithmetic interface, PDE interface, or problem-specific conclusion is introduced.

A.18. Provenance and status

The constructible persistence setting, interval decomposition, barcode uniqueness, and barcode stability belong to standard one-parameter persistence theory over a field. The positive-threshold repair, the explicit non-Serre warning, the barcode-level interpretation of T_τ^{bar} , and the fixed windowed evaluation order are part of the AK v18.1 Core discipline.

Status labels for Appendix A. *Foundation definitions:* Definitions [1.2](#), [1.3](#), [1.6](#), [1.10](#), [1.11](#), [1.14](#), [1.15](#), [1.17](#), [1.22](#), [1.29](#), [1.32](#), [1.37](#), [1.40](#), [1.43](#), [1.44](#), [1.45](#), [1.47](#), [1.49](#), [1.50](#), [1.52](#), [1.53](#).

Foundation theorems/propositions: Theorem [1.12](#), Proposition [1.8](#), Proposition [1.18](#), Corollary [1.19](#), Corollary [1.20](#), Proposition [1.24](#), Proposition [1.25](#), Proposition [1.26](#), Theorem [1.30](#), Proposition [1.33](#), Proposition [1.35](#), Proposition [1.41](#), Proposition [1.46](#), Theorem [1.55](#).

Operational cautions: Remarks [1.9](#), [1.23](#), [1.31](#), [1.36](#), [1.38](#), [1.42](#), [1.48](#), [1.51](#), [1.54](#).

Explicit exclusions: Section A.17.

2 Appendix B: Filtered Representatives of Windowed Barcode Collapse

B.1. Scope and dependence on the repaired Core

This appendix supplies the filtered-level foundation for Chapters 2 and 3 after the v18.1 Core repair.

Chapter 2 and Appendix A no longer treat positive-threshold bar deletion as a global exact Serre reflector on all of $\text{Pers}_k^{\text{cons}}$. Instead, the Core uses the barcode-level threshold deletion operation

$$T_\tau^{\text{bar}}$$

and the fixed windowed evaluation order

$$\text{Eval}_{i,W,\tau}(F) := T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

The purpose of this appendix is to specify how such windowed collapsed persistence profiles may be represented by filtered chain complexes when filtered or derived constructions are needed.

The central rule of this appendix is:

A filtered representative does not define collapse. It only represents a declared windowed collapsed persistence profile.

Thus, in the general Core regime, the primary object is not a global functor

$$C_{\tau} : \text{FiltCh}(\cdot)k \rightarrow \text{FiltCh}(\cdot)k,$$

but a windowed admissible representative

$$C_{\tau,W}F$$

whose monitored persistence agrees with

$$\text{Eval}_{i,W,\tau}(F).$$

Remark 2.1 (Appendix B does not enlarge the Core). This appendix does not introduce new Core theorems beyond the representative discipline needed by Chapters 2 and 3. It does not restore the old global exact-collapse claim. It only explains how barcode-level windowed collapse data may be represented at the filtered-chain level when such representatives are available.

B.2. Filtered chain complexes and monitored persistence

Definition 2.2 (Finite-type filtered chain complex). A *finite-type filtered chain complex* over k is a bounded chain complex

$$C_{\bullet}$$

of finite-dimensional k -vector spaces equipped with an increasing filtration

$$F^t C_{\bullet} \subseteq F^{t'} C_{\bullet} \quad (t \leq t')$$

such that:

- (i) each $F^t C_{\bullet}$ is a subcomplex;
- (ii) the filtration is exhaustive and left-bounded;
- (iii) for each monitored degree i , the persistence module

$$\mathbf{P}_i(F) : t \mapsto H_i(F^t C_{\bullet})$$

lies in $\text{Pers}_k^{\text{cons}}$.

The category of such objects and filtration-preserving chain maps is denoted

$$\text{FiltCh}(\cdot)k.$$

Definition 2.3 (Persistence functor). For each monitored homological degree i , define

$$\mathbf{P}_i : \text{FiltCh}(\cdot)k \longrightarrow \text{Pers}_k^{\text{cons}}$$

by

$$\mathbf{P}_i(F)(t) := H_i(F^t C_{\bullet}).$$

Remark 2.4 (Monitored-degree convention). The Core uses $\mathbf{P}_i$ degree-wise and only for declared monitored degrees. No statement in this appendix controls all homological degrees unless the monitored range is explicitly declared to be the full range.

B.3. Filtered quasi-isomorphism

Definition 2.5 (Filtered quasi-isomorphism). A filtration-preserving chain map

$$f : F \rightarrow G$$

in $\text{FiltCh}(()k)$ is called a *filtered quasi-isomorphism on the monitored range* if, for every monitored degree i , it induces an isomorphism

$$\mathbf{P}_i(F) \cong \mathbf{P}_i(G)$$

in $\text{Pers}_k^{\text{cons}}$.

Definition 2.6 (Representative equivalence). Two filtered objects F and G are *representatively equivalent* for a declared Core run if their monitored persistence profiles agree:

$$\mathbf{P}_i(F) \cong \mathbf{P}_i(G)$$

for all monitored degrees i , under the declared barcode or filtered quasi-isomorphism policy.

Remark 2.7 (Why filtered quasi-isomorphism is sufficient). The Core reads filtered objects through monitored persistence profiles and, when needed, through realizations compatible with those profiles. Strict equality of filtered complexes is therefore stronger than necessary. Filtered quasi-isomorphism, or the declared representative equivalence, is the correct equality notion for Core gate purposes.

Definition 2.8 (Localized representative category). Let

$$\text{Ho}_{\text{f.q.i.}}(\text{FiltCh}(()k))$$

denote the localization of $\text{FiltCh}(()k)$ at filtered quasi-isomorphisms on the monitored range. Whenever filtered representatives are compared without strict implementation data, the comparison is read in this localized category.

B.4. Windowed collapsed persistence profiles

Recall from Chapter 2 and Appendix A that the repaired Core evaluates monitored persistence data by

$$\text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Definition 2.9 (Windowed collapsed persistence profile). For $F \in \text{FiltCh}(()k)$, monitored degree i , right-open window W , and threshold $\tau > 0$, the *windowed collapsed persistence profile* is

$$\mathcal{P}_{i,W,\tau}(F) := \text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Remark 2.10 (The profile is primary). The profile $\mathcal{P}_{i,W,\tau}(F)$ is the primary Core object. A filtered representative is secondary data used only when a filtered-chain or derived-category construction is required.

Definition 2.11 (Profile family over monitored degrees). For a monitored degree set I_{mon} , define the windowed collapsed profile family

$$\mathcal{P}_{W,\tau}(F) := \{\mathcal{P}_{i,W,\tau}(F)\}_{i \in I_{\text{mon}}}.$$

B.5. Admissible filtered representatives

Definition 2.12 (Admissible filtered representative). Let $F \in \text{FiltCh}(\cdot)k$, let W be a right-open window, and let $\tau > 0$. A filtered object

$$C_{\tau,W}F \in \text{FiltCh}(\cdot)k$$

is called an *admissible filtered representative* of the windowed collapsed profile of F if, for every monitored degree i ,

$$\mathbf{P}_i(C_{\tau,W}F) \cong \mathcal{P}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F))$$

under the declared profile-equivalence policy.

Remark 2.13 (Representative, not functor). The notation $C_{\tau,W}F$ denotes an admissible representative of a declared profile. It does not, by itself, assert that $C_{\tau,W}$ is a globally defined functor on $\text{FiltCh}(\cdot)k$.

Definition 2.14 (Representative compatibility). An admissible representative $C_{\tau,W}F$ is said to be *compatible* with the repaired Core evaluation if the following diagrammatic statement holds at the monitored persistence level:

$$\mathbf{P}_i(C_{\tau,W}F) \cong T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F))$$

for all monitored degrees i .

Proposition 2.15 (Compatibility is exactly the representative condition). *A filtered object $C_{\tau,W}F$ is an admissible filtered representative if and only if it satisfies representative compatibility for all monitored degrees.*

Proof. This is a restatement of Definition 2.12. □

B.6. Implementable range

Definition 2.16 (Implementable representative range). The *implementable representative range* is the declared class

$$\text{FiltCh}(\cdot)^{\text{rep}}(k) \subseteq \text{FiltCh}(\cdot)k$$

of finite-type filtered chain complexes for which the following data can be supplied for the relevant windows and thresholds:

- (i) constructible monitored persistence profiles $\mathbf{P}_i(F)$;
- (ii) locally finite barcodes on all declared windows;
- (iii) windowed collapsed profiles
$$\mathcal{P}_{i,W,\tau}(F)$$
for all monitored i, W, τ ;
- (iv) at least one admissible filtered representative $C_{\tau,W}F$ when filtered-level or derived-level arguments are invoked;
- (v) representative equivalence data, normally filtered quasi-isomorphism or monitored barcode equivalence;
- (vi) artifact references sufficient to reconstruct the representative choice.

Remark 2.17 (Why an implementable range is needed). A barcode-level collapsed profile need not come with a canonical filtered chain complex representative. The implementable representative range is the regime in which representatives are available and auditable. Outside this range, the Core may still evaluate

$$\text{Eval}_{i,W,\tau}(F)$$

at the barcode-profile level, but it may not invoke filtered or derived statements requiring $C_{\tau,W}F$.

Axiom 2.1 (Representative existence in the implementable range). *For every $F \in \text{FiltCh}(\text{rep}(k))$, every declared window W , every threshold $\tau > 0$, and every monitored degree set I_{mon} , there exists an admissible filtered representative*

$$C_{\tau,W}F$$

of the profile family

$$\mathcal{P}_{W,\tau}(F).$$

Remark 2.18 (Status of the representative existence axiom). Axiom 2.1 is an operational existence assumption for the implementable representative range. It is not a theorem that every barcode-level collapse has a unique or canonical filtered representative. When a concrete model constructs such representatives, the axiom is discharged for that model.

B.7. Non-uniqueness and representative equivalence

Proposition 2.19 (Non-uniqueness of filtered representatives). *An admissible filtered representative $C_{\tau,W}F$, when it exists, is generally not unique as a strict filtered chain complex.*

Proof. The profile

$$\mathcal{P}_{i,W,\tau}(F)$$

specifies monitored persistence data, not a unique filtered chain complex. Different filtered complexes may realize the same monitored persistence profile. Thus strict uniqueness is not expected. $\square$

Definition 2.20 (Admissible representative choice). *An admissible representative choice for (F, W, τ) consists of:*

- (i) a filtered object $C_{\tau,W}F$;
- (ii) the monitored degree set I_{mon} ;
- (iii) isomorphisms

$$\mathbf{P}_i(C_{\tau,W}F) \cong \mathcal{P}_{i,W,\tau}(F)$$

for all $i \in I_{\text{mon}}$;

- (iv) an equivalence policy for comparing alternative representatives;
- (v) artifact references for the representative construction.

Proposition 2.21 (Representative invariance of Core persistence clauses). *If $C_{\tau,W}F$ and $C'_{\tau,W}F$ are admissible representatives of the same profile family $\mathcal{P}_{W,\tau}(F)$, then all persistence-level Core clauses depending only on monitored $\mathbf{P}_i$ are invariant under replacing $C_{\tau,W}F$ by $C'_{\tau,W}F$.*

Proof. For each monitored degree i ,

$$\mathbf{P}_i(C_{\tau,W}F) \cong \mathcal{P}_{i,W,\tau}(F) \cong \mathbf{P}_i(C'_{\tau,W}F).$$

Therefore any Core clause depending only on these monitored persistence profiles has the same value for both representatives. $\square$

Remark 2.22 (No hidden canonicity). The notation $C_{\tau,W}F$ should never be read as a hidden canonical object. It denotes an admissible representative class, not a unique strict filtered complex.

B.8. Global C_τ notation and its restricted use

The repaired Core avoids using a global strict C_τ by default. However, some exact-admissible or implementation-specific regimes may supply a global representative construction.

Definition 2.23 (Global representative construction). A *global representative construction* at threshold τ is a declared assignment

$$F \mapsto C_\tau F$$

such that, for every monitored degree i ,

$$\mathbf{P}_i(C_\tau F) \cong T_\tau^{\text{bar}}(\mathbf{P}_i(F))$$

at the barcode-profile level.

Remark 2.24 (Global representative construction is not default). Definition 2.23 is optional. The default Core object remains $C_{\tau,W}F$, not $C_\tau F$. A global representative construction may be used only when explicitly declared.

Definition 2.25 (Window compatibility of a global representative). A global representative construction C_τ is *window-compatible on W* if, for all monitored degrees i ,

$$W_W \mathbf{P}_i(C_\tau F) \cong T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Equivalently, the window restriction of the global representative realizes the repaired windowed profile.

Proposition 2.26 (Restriction of a global representative under window compatibility). *If C_τ is window-compatible on W , then $(C_\tau F)|_W$ is an admissible representative of $C_{\tau,W}F$.*

Proof. By window compatibility,

$$\mathbf{P}_i((C_\tau F)|_W) \cong W_W \mathbf{P}_i(C_\tau F) \cong T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)) = \mathcal{P}_{i,W,\tau}(F)$$

for all monitored degrees i . Thus $(C_\tau F)|_W$ satisfies the admissible representative condition. $\square$

Remark 2.27 (Why $C_{\tau,W}F \text{ eq } (C_\tau F)|_W$ by default). In general,

$$W_W T_\tau^{\text{bar}} \not\cong T_\tau^{\text{bar}} W_W.$$

Therefore a representative of global threshold deletion, even if supplied, need not restrict to a representative of the repaired windowed evaluation. The identity

$$C_{\tau,W}F = (C_\tau F)|_W$$

is forbidden unless window compatibility is declared.

B.9. Idempotence at the representative level

Proposition 2.28 (Windowed representative idempotence). *Let $C_{\tau,W}F$ be an admissible representative of $\mathcal{P}_{W,\tau}(F)$. If the second application is interpreted using the same declared profile and representative policy, then*

$$C_{\tau,W}(C_{\tau,W}F)$$

is representatively equivalent to

$$C_{\tau,W}F.$$

Proof. For every monitored degree i ,

$$\mathbf{P}_i(C_{\tau,W}F) \cong T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

This profile contains no finite bar of window-visible length at most τ . Applying the same windowed threshold deletion again removes no additional monitored bars. Therefore the monitored profiles agree, and the two representatives are equivalent under the declared representative policy. $\square$

Remark 2.29 (Idempotence is profile-level). The idempotence statement is an idempotence of windowed collapsed profiles and their representatives. It is not a statement that a globally strict functor C_{τ} is idempotent on all of $\text{FiltCh}(()k)$.

B.10. Threshold composition for representatives

Proposition 2.30 (Windowed threshold composition). *Let $\tau, \sigma > 0$, and suppose all relevant representatives exist in the implementable representative range. Then the representative obtained by applying threshold τ and then threshold σ is representatively equivalent to a representative at threshold $\max\{\tau, \sigma\}$:*

$$C_{\sigma,W}(C_{\tau,W}F) \sim_{\text{rep}} C_{\max\{\tau,\sigma\},W}F.$$

Proof. For every monitored degree i , the corresponding profile calculation is

$$T_{\sigma}^{\text{bar}} T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)) \cong T_{\max\{\tau,\sigma\}}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Thus the two representatives realize the same monitored windowed profile and are representatively equivalent. $\square$

Remark 2.31 (Threshold sweeps). Threshold sweeps use retained-profile comparisons

$$T_{\tau}^{\text{bar}} \rightsquigarrow T_{\sigma}^{\text{bar}} \quad (0 < \tau \leq \sigma)$$

inside each declared window. No strict nested family of filtered complexes is asserted unless separately constructed.

B.11. Weak functoriality and morphism data

Because T_{τ}^{bar} is not asserted to be a global exact functor in the general Core, morphism-level statements for representatives require declared comparison data.

Definition 2.32 (Representative morphism datum). Let

$$f : F \rightarrow G$$

be a filtration-preserving map, and let $C_{\tau,W}F$, $C_{\tau,W}G$ be admissible representatives. A *representative morphism datum* for f consists of a morphism in the localized representative category

$$C_{\tau,W}f : C_{\tau,W}F \rightarrow C_{\tau,W}G$$

whose monitored persistence-level action agrees with the declared barcode-profile comparison induced by f , when such a comparison is available.

Definition 2.33 (Weak representative functoriality). A representative construction is *weakly functorial* on a declared class of morphisms if every morphism in that class is equipped with representative morphism data compatible with monitored persistence profiles and composition up to representative equivalence.

Proposition 2.34 (Weak functoriality is conditional). *Weak representative functoriality holds only relative to a declared class of morphisms and declared representative morphism data. It is not automatic in the general Core.*

Proof. Without a global strict functorial collapse operation, a morphism $f : F \rightarrow G$ does not automatically determine a strict morphism between arbitrary representative choices. Such a morphism exists only after representative choices and compatible profile comparison data are supplied. Therefore weak functoriality is conditional on the declared data. $\square$

Remark 2.35 (No strict global functor). The Core does not assert a strict globally defined endofunctor

$$C_{\tau} : \text{FiltCh}(\cdot)k \rightarrow \text{FiltCh}(\cdot)k$$

nor a strict globally defined bifunctorial construction

$$(F, W) \mapsto C_{\tau,W}F.$$

All such strict constructions are implementation-specific unless an exact-admissible regime supplies them.

B.12. Compatibility with realization

Chapter 3 applies a realization functor

$$\mathcal{R} : \text{FiltCh}(\cdot)k \rightarrow D^b(k\text{-mod})$$

to an admissible representative.

Definition 2.36 (Realization-compatible representative). An admissible representative $C_{\tau,W}F$ is *realization-compatible* if:

- (i) $\mathcal{R}(C_{\tau,W}F)$ is defined in $D^b(k\text{-mod})$;
- (ii) representatively equivalent choices yield isomorphic realized objects;
- (iii) the eligible realization condition can be evaluated on $\mathcal{R}(C_{\tau,W}F)$;
- (iv) if the Ext-clause is invoked, the amplitude and edge-realization conditions are declared.

Proposition 2.37 (Representative invariance of realization). *If $C_{\tau,W}F$ and $C'_{\tau,W}F$ are representatively equivalent admissible representatives, and $\mathcal{R}$ is compatible with representative equivalence, then*

$$\mathcal{R}(C_{\tau,W}F) \cong \mathcal{R}(C'_{\tau,W}F)$$

in $D^b(k\text{-mod})$.

Proof. This is precisely compatibility of the realization functor with the declared representative equivalence. Such compatibility is part of the eligible realization data required when Ext is invoked. $\square$

Remark 2.38 (Why this matters for the PH–Ext bridge). The one-way bridge

$$PH_1 \Rightarrow \text{Ext}^1$$

is applied to

$$\mathcal{R}(C_{\tau,W}F).$$

Therefore the realized object must be insensitive to admissible representative choices, at least up to isomorphism in $D^b(k\text{-mod})$.

B.13. Relation to $B\text{-Gate}^+$

Definition 2.39 (Representative input to $B\text{-Gate}^+$). A representative input to $B\text{-Gate}^+$ consists of:

- (i) a filtered object F ;
- (ii) a declared window W ;
- (iii) a threshold τ ;
- (iv) monitored degrees I_{mon} ;
- (v) the collapsed profiles

$$\mathcal{P}_{i,W,\tau}(F)$$

for $i \in I_{\text{mon}}$;

- (vi) an admissible representative $C_{\tau,W}F$, if any filtered or derived clause is invoked;
- (vii) realization-compatible data if the Ext-clause is invoked.

Definition 2.40 (Topological clause via representative). The topological clause of $B\text{-Gate}^+$ on (F, W, τ) may be read as

$$PH_1(C_{\tau,W}F) = 0$$

only when $C_{\tau,W}F$ realizes

$$\mathcal{P}_{1,W,\tau}(F).$$

Equivalently, at the profile level, it is the vanishing of

$$\text{Eval}_{1,W,\tau}(F).$$

Definition 2.41 (Categorical clause via representative). The categorical clause of $B\text{-Gate}^+$ may be read as

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0$$

only when $C_{\tau,W}F$ is an admissible realization-compatible representative and the eligible regime of Chapter 3 is verified.

Remark 2.42 (No representative, no Ext-clause). If only the barcode-level profile $\text{Eval}_{i,W,\tau}(F)$ is available, the Core may evaluate persistence-level clauses. It may not invoke the Ext-clause unless an admissible realization-compatible representative is supplied.

B.14. Tower representatives and Type IV input

Definition 2.43 (Representative tower). Let

$$F_0 \rightarrow F_1 \rightarrow F_2 \rightarrow \dots$$

be a directed tower in the declared regime. A *representative tower* for (i, W, τ) consists of admissible representatives

$$C_{\tau, W} F_n$$

for the windowed collapsed profiles

$$\mathcal{P}_{i, W, \tau}(F_n),$$

together with representative morphism data between consecutive levels.

Definition 2.44 (Representative tower comparison artifact). A representative tower comparison artifact is a declared comparison

$$\phi_{i, W, \tau} : \text{ColimProfile}_{i, W, \tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i, W, \tau}(F_\infty)$$

together with enough representative or persistence-profile data to compute the monitored kernel and cokernel defects.

Remark 2.45 (Type IV input). Type IV diagnostics do not follow from the existence of representatives alone. They require a declared tower comparison artifact. Representatives merely provide one way to realize or compute the profile data entering that artifact.

B.15. Certificate and manifest requirements

Definition 2.46 (Representative declaration in a certificate record). A certificate record using an admissible representative must declare:

- (i) the threshold τ ;
- (ii) the window W ;
- (iii) the monitored degrees I_{mon} ;
- (iv) the fixed evaluation order

$$\mathbf{P}_i \rightarrow W_W \mathbf{P}_i \rightarrow T_\tau^{\text{bar}}(W_W \mathbf{P}_i);$$
- (v) the collapsed profiles $\mathcal{P}_{i, W, \tau}(F)$;
- (vi) the representative $C_{\tau, W} F$, if used;
- (vii) the representative equivalence policy;
- (viii) realization compatibility, if Ext is invoked;
- (ix) morphism or tower comparison artifacts, if Type IV diagnostics are invoked;
- (x) artifact references sufficient for audit reconstruction.

Remark 2.47 (Manifest-level role). The representative declaration is not a cosmetic implementation detail. When a representative is used in a Core gate, bridge, or diagnostic, it is part of the audit-readable proof object.

B.16. Summary of filtered-representative foundations

Theorem 2.48 (Appendix B filtered-representative package). *Within the declared implementable representative range, the following package is available.*

(i) *The primary Core object is the windowed collapsed persistence profile*

$$\mathcal{P}_{i,W,\tau}(F) = \text{Eval}_{i,W,\tau}(F).$$

(ii) *An admissible representative $C_{\tau,W}F$ realizes this profile at the monitored persistence level:*

$$\mathbf{P}_i(C_{\tau,W}F) \cong \mathcal{P}_{i,W,\tau}(F).$$

(iii) *Representatives are generally non-unique.*

(iv) *Representative equivalence preserves all persistence-level Core clauses.*

(v) *A global representative construction $C_{\tau}F$ is optional and not part of the default Core.*

(vi) *$C_{\tau,W}F$ is not generally equal to $(C_{\tau}F)|_W$.*

(vii) *Representative idempotence and threshold composition hold at the monitored profile level.*

(viii) *Weak functoriality requires declared representative morphism data.*

(ix) *Ext-clauses require realization-compatible representatives.*

(x) *Type IV diagnostics require tower comparison artifacts.*

(xi) *Certificate records must declare representative, realization, and comparison data whenever those structures are used.*

Proof. Items (i)–(ii) follow from Definitions 2.9 and 2.12. Item (iii) is Proposition 2.19. Item (iv) is Proposition 2.21. Items (v)–(vi) follow from the restricted status of global representatives and the non-commutation of window restriction with threshold deletion. Item (vii) follows from Propositions 2.28 and 2.30. Item (viii) follows from Proposition 2.34. Item (ix) follows from Definition 2.36. Item (x) follows from Definition 2.44. Item (xi) follows from Definition 2.46. $\square$

B.17. Explicit exclusions

The following are not asserted in Appendix B.

(i) No strict globally defined endofunctor

$$C_{\tau} : \text{FiltCh}(()k) \rightarrow \text{FiltCh}(()k)$$

is asserted by default.

(ii) No strict globally defined functor

$$(F, W) \mapsto C_{\tau,W}F$$

is asserted by default.

(iii) No uniqueness of $C_{\tau,W}F$ as a strict filtered chain complex is asserted.

(iv) No unconditional identity

$$C_{\tau, W}F = (C_{\tau}F)|_W$$

is asserted.

(v) No global exactness of positive-threshold collapse is restored here.

(vi) No Serre localization claim for positive-threshold short bars is restored here.

(vii) No AWFS structure is introduced.

(viii) No soft-commuting calculus is introduced.

(ix) No Mirror/Transfer 2-cell control is introduced.

(x) No Search-layer or AI-platform mechanism is used to define a representative.

(xi) No problem-specific arithmetic, PDE, geometric, or categorical realization is introduced.

(xii) No representative is asserted outside the declared implementable representative range.

(xiii) No Ext-clause is invoked without a realization-compatible representative and the eligible regime.

B.18. Provenance and status

The persistence-level barcode collapse T_{τ}^{bar} , the fixed evaluation order

$$\text{Eval}_{i, W, \tau}(F),$$

and the non-commutation of window restriction with threshold deletion are supplied by Chapter 2 and Appendix A.

The present appendix explains how those persistence-profile objects may be represented by filtered chain complexes in an implementable range. The filtered representative is a representative mechanism, not an independent collapse operation.

Status labels for Appendix B. *Foundation definitions:* Definitions [2.2](#), [2.3](#), [2.5](#), [2.6](#), [2.8](#), [2.9](#), [2.11](#), [2.12](#), [2.14](#), [2.16](#), [2.20](#), [2.23](#), [2.25](#), [2.32](#), [2.33](#), [2.36](#), [2.39](#), [2.40](#), [2.41](#), [2.43](#), [2.44](#), [2.46](#).

Foundation axioms/propositions/theorems: Axiom [2.1](#), Propositions [2.15](#), [2.19](#), [2.21](#), [2.26](#), [2.28](#), [2.30](#), [2.34](#), [2.37](#), Theorem [2.48](#).

Operational cautions: Remarks [2.1](#), [2.10](#), [2.13](#), [2.17](#), [2.18](#), [2.22](#), [2.24](#), [2.27](#), [2.29](#), [2.31](#), [2.35](#), [2.38](#), [2.42](#), [2.45](#), [2.47](#).

Explicit exclusions: Section B.17.

3 Appendix C: Eligible Realization and Ext-edge Details

C.1. Scope and dependence on the repaired Core

This appendix supplies the derived-category and eligibility details supporting Chapter 3 after the v18.1 Core repair.

The purpose is to justify the one-way bridge

$$\text{PH}_1 \Rightarrow \text{Ext}^1$$

inside the declared eligible regime.

All statements in this appendix are read relative to the repaired windowed collapse discipline:

$$\text{Eval}_{i,W,\tau}(F) := T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

The filtered object

$$C_{\tau,W}F$$

is an admissible representative of this repaired windowed collapsed profile. It is not a global strict collapse functor and is not assumed to be obtained by restricting a global $C_{\tau}F$.

More precisely, this appendix expands the following points:

- (i) the derived category setting;
- (ii) the meaning of the eligible realization regime;
- (iii) the role of amplitude $[-1, 0]$;
- (iv) the derived splitting over a field;
- (v) the Ext^1 -edge identification

$$\text{Ext}^1(A, k) \cong \text{Hom}_k(H^{-1}(A), k);$$

- (vi) the edge-realization condition linking the repaired residual degree-1 persistence profile to H^{-1} ;
- (vii) the proof of the one-way bridge;
- (viii) representative invariance;
- (ix) the explicit exclusion of any reverse bridge.

Remark 3.1 (Appendix C does not enlarge the Core). This appendix does not introduce a new bridge principle. It only supplies the details behind the Chapter 3 statement that, in the eligible regime,

$$\text{Eval}_{1,W,\tau}(F) = 0 \implies \text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

Equivalently, if $C_{\tau,W}F$ is an admissible representative, the same implication may be read as

$$\text{PH}_1(C_{\tau,W}F) = 0 \implies \text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

No converse implication is asserted here.

C.2. Derived category setting

Fix a field k . Let

$$D^b(k\text{-mod})$$

denote the bounded derived category of finite-dimensional k -vector spaces.

Definition 3.2 (Bounded derived object). An object

$$A \in D^b(k\text{-mod})$$

is represented by a bounded cochain complex of finite-dimensional k -vector spaces. Its cohomology objects are denoted

$$H^j(A).$$

Definition 3.3 (Amplitude). An object $A \in D^b(k\text{-mod})$ has *amplitude contained in* $[a, b]$ if

$$H^j(A) = 0 \quad \text{for all } j \notin [a, b].$$

We write

$$A \in D^{[a,b]}(k\text{-mod})$$

for this condition.

Definition 3.4 (Two-term eligible amplitude). An object

$$A \in D^b(k\text{-mod})$$

has *eligible two-term amplitude* if

$$A \in D^{[-1,0]}(k\text{-mod}).$$

Equivalently,

$$H^j(A) = 0 \quad \text{for } j \neq -1, 0.$$

Remark 3.5 (Why amplitude $[-1, 0]$ is used). The amplitude restriction $[-1, 0]$ isolates a single possible Ext^1 -edge. In this regime, the obstruction group

$$\text{Ext}^1(A, k)$$

is controlled by the -1 -st cohomology object $H^{-1}(A)$. This is exactly the edge needed for the one-way bridge.

C.3. Repaired windowed collapsed profiles

Definition 3.6 (Repaired windowed collapsed profile). For $F \in \text{FiltCh}(()k)$, monitored degree i , right-open window $W = [u, u')$, and threshold $\tau > 0$, define

$$\text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

This is the repaired Core persistence profile used in all gate, bridge, and realization statements.

Definition 3.7 (Admissible representative). An object

$$C_{\tau,W}F \in \text{FiltCh}(()k)$$

is an *admissible representative* of the repaired windowed collapsed profile of F if, for every monitored degree i ,

$$\mathbf{P}_i(C_{\tau,W}F) \cong \text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F))$$

under the declared representative equivalence policy.

Remark 3.8 (Representative, not global collapse). The notation $C_{\tau,W}F$ does not denote a globally strict collapse functor. It denotes an admissible filtered representative of a repaired windowed collapsed persistence profile. No identity

$$C_{\tau,W}F = (C_\tau F)|_W$$

is assumed.

Remark 3.9 (Dependence on Appendix B). Appendix B supplies the representative discipline for $C_{\tau,W}F$. The present appendix uses that discipline only to pass from repaired persistence profiles to realized derived objects. Thus the derived-category bridge is applied to

$$\mathcal{R}(C_{\tau,W}F),$$

not to an unspecified global collapse object.

C.4. Realization functor

Definition 3.10 (Realization functor). A *realization functor* is a functor

$$\mathcal{R} : \text{FiltCh}(\cdot)k \longrightarrow D^b(k\text{-mod})$$

or, more generally, a functor defined on a declared implementable representative subcategory

$$\text{FiltCh}(\cdot)\text{rep}(k) \subseteq \text{FiltCh}(\cdot)k,$$

such that admissibly equivalent representatives are sent to isomorphic objects in

$$D^b(k\text{-mod}).$$

Definition 3.11 (Realization-compatible representative). Let $C_{\tau,W}F$ be an admissible representative in the sense of Definition 3.7. It is *realization-compatible* if

$$\mathcal{R}(C_{\tau,W}F)$$

is well-defined in $D^b(k\text{-mod})$ and is invariant, up to isomorphism in $D^b(k\text{-mod})$, under admissible representative equivalence.

Remark 3.12 (Why realization compatibility is required). The Ext^1 -clause is a statement about the realized derived object. If different admissible representatives could lead to non-isomorphic realized objects, the Ext^1 -verdict would depend on an untracked representative choice. Realization compatibility prevents this.

C.5. Eligible realization regime

Definition 3.13 (Eligible realized object). Let $F \in \text{FiltCh}(\cdot)\text{rep}(k)$, let $W = [u, u']$ be a right-open window, and let $\tau > 0$. Given a realization-compatible admissible representative $C_{\tau,W}F$, define the realized repaired windowed collapsed object by

$$A_{F,W,\tau} := \mathcal{R}(C_{\tau,W}F).$$

Definition 3.14 (Eligible regime). The triple

$$(F; W, \tau)$$

together with an admissible representative $C_{\tau,W}F$ is said to be *eligible* if all of the following hold:

- (i) $F \in \text{FiltCh}(\cdot)\text{rep}(k)$;
- (ii) $C_{\tau,W}F$ is an admissible representative of the repaired windowed collapsed profile;
- (iii) $C_{\tau,W}F$ is realization-compatible;
- (iv) the realized object satisfies

$$A_{F,W,\tau} = \mathcal{R}(C_{\tau,W}F) \in D^{[-1,0]}(k\text{-mod});$$

- (v) the edge-realization condition of Definition 3.17 holds.

Remark 3.15 (Eligibility is a gate condition). Eligibility is not merely a convenience. It is the condition that licenses the use of the Ext^1 -edge. Without eligibility, the categorical clause of B-Gate^+ is not available.

C.6. Edge-realization condition

Definition 3.16 (Residual degree-1 persistence edge). For a filtered object F , window W , and threshold τ , the *residual degree-1 persistence edge* is the monitored vector-space edge represented by the repaired windowed collapsed degree-1 profile

$$\text{Eval}_{1,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_1(F)).$$

We denote this edge by

$$\mathcal{H}_1^{\tau,W}(F).$$

Definition 3.17 (Edge-realization condition). An eligible realization satisfies the *edge-realization condition* if there is a declared identification

$$H^{-1}(A_{F,W,\tau}) \cong \mathcal{H}_1^{\tau,W}(F),$$

where

$$A_{F,W,\tau} = \mathcal{R}(C_{\tau,W}F).$$

In particular,

$$\text{Eval}_{1,W,\tau}(F) = 0 \implies H^{-1}(A_{F,W,\tau}) = 0.$$

Equivalently, if $C_{\tau,W}F$ is an admissible representative,

$$\text{PH}_1(C_{\tau,W}F) = 0 \implies H^{-1}(A_{F,W,\tau}) = 0.$$

Remark 3.18 (Meaning of the edge-realization condition). The amplitude condition alone is not enough to connect repaired degree-1 persistence to Ext^1 . One must also know that the repaired residual degree-1 persistence edge is represented by the -1 -st cohomology of the realized object. This is exactly what the edge-realization condition asserts.

Remark 3.19 (No raw persistence-to-derived shortcut). The edge-realization condition applies to the repaired profile

$$\text{Eval}_{1,W,\tau}(F),$$

not to the raw persistence module $\mathbf{P}_1(F)$. Thus the bridge is after-window and after-collapse.

C.7. Splitting over a field

Lemma 3.20 (Semisimplicity over a field). *The abelian category*

$$k\text{-mod}$$

of finite-dimensional k -vector spaces is semisimple. Equivalently, every short exact sequence of finite-dimensional k -vector spaces splits.

Proof. Every finite-dimensional vector space over a field is free, hence projective and injective. Therefore every short exact sequence of finite-dimensional k -vector spaces splits. $\square$

Lemma 3.21 (Derived splitting over a field). *Let*

$$A \in D^b(k\text{-mod}).$$

Then A is isomorphic in $D^b(k\text{-mod})$ to the direct sum of its cohomology objects:

$$A \cong \bigoplus_{j \in \mathbb{Z}} H^j(A)[-j].$$

Proof. Choose a bounded complex representing A . By Lemma 3.20, all short exact sequences of cycles, boundaries, and cohomology split degree-wise. Thus the complex is quasi-isomorphic to its cohomology complex with zero differential. This gives the decomposition

$$A \cong \bigoplus_j H^j(A)[-j]$$

in the bounded derived category. □

Corollary 3.22 (Two-term splitting). *If*

$$A \in D^{[-1,0]}(k\text{-mod}),$$

then

$$A \cong H^{-1}(A)[1] \oplus H^0(A).$$

Proof. Apply Lemma 3.21 and use the vanishing

$$H^j(A) = 0 \quad (j \neq -1, 0).$$

□

C.8. The Ext^1 -edge

Definition 3.23 (Ext^1 in the derived category). For

$$A \in D^b(k\text{-mod}),$$

define

$$\text{Ext}^1(A, k) := \text{Hom}_{D^b(k\text{-mod})}(A, k[1]).$$

Theorem 3.24 (Ext^1 -edge identification). *Let*

$$A \in D^{[-1,0]}(k\text{-mod}).$$

Then there is a natural identification

$$\text{Ext}^1(A, k) \cong \text{Hom}_k(H^{-1}(A), k).$$

Proof. By Corollary 3.22,

$$A \cong H^{-1}(A)[1] \oplus H^0(A).$$

Therefore

$$\text{Ext}^1(A, k) = \text{Hom}_{D^b(k\text{-mod})}(A, k[1])$$

is naturally isomorphic to

$$\text{Hom}_{D^b(k\text{-mod})}(H^{-1}(A)[1], k[1]) \oplus \text{Hom}_{D^b(k\text{-mod})}(H^0(A), k[1]).$$

The first summand is

$$\text{Hom}_k(H^{-1}(A), k).$$

The second summand is

$$\text{Ext}_k^1(H^0(A), k),$$

which vanishes because $k\text{-mod}$ is semisimple. Hence

$$\text{Ext}^1(A, k) \cong \text{Hom}_k(H^{-1}(A), k).$$

□

Corollary 3.25 (Edge vanishing criterion). *If*

$$A \in D^{[-1,0]}(k\text{-mod})$$

and

$$H^{-1}(A) = 0,$$

then

$$\text{Ext}^1(A, k) = 0.$$

Proof. Immediate from Theorem 3.24. □

Remark 3.26 (No reverse information). The identification

$$\text{Ext}^1(A, k) \cong \text{Hom}_k(H^{-1}(A), k)$$

is used only to discharge Ext^1 from vanishing of H^{-1} . It is not used to reconstruct PH_1 from Ext^1 , and it does not license any global reverse bridge.

C.9. From repaired PH_1 to the edge

Proposition 3.27 (Repaired degree-1 vanishing implies edge vanishing). *Let $(F; W, \tau)$ be eligible. If*

$$\text{Eval}_{1,W,\tau}(F) = 0,$$

equivalently, if for an admissible representative

$$\text{PH}_1(C_{\tau,W}F) = 0,$$

then

$$H^{-1}(A_{F,W,\tau}) = 0.$$

Proof. By the edge-realization condition,

$$H^{-1}(A_{F,W,\tau}) \cong \mathcal{H}_1^{\tau,W}(F).$$

The object $\mathcal{H}_1^{\tau,W}(F)$ is the residual degree-1 persistence edge represented by

$$\text{Eval}_{1,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_1(F)).$$

Thus the vanishing

$$\text{Eval}_{1,W,\tau}(F) = 0$$

means that this residual degree-1 edge is zero. Hence

$$H^{-1}(A_{F,W,\tau}) = 0.$$

If an admissible representative $C_{\tau,W}F$ is supplied, the equivalent statement

$$\text{PH}_1(C_{\tau,W}F) = 0$$

expresses the same vanishing at the representative level. □

Remark 3.28 (Where the topological input enters). The only topological input to the one-way bridge is the vanishing of the repaired windowed collapsed degree-1 persistence edge. Everything after that point is derived-category bookkeeping in amplitude $[-1, 0]$.

C.10. The one-way bridge

Theorem 3.29 (One-way bridge). *Let $(F; W, \tau)$ be eligible. If*

$$\text{Eval}_{1,W,\tau}(F) = 0,$$

equivalently, if for an admissible representative

$$\text{PH}_1(C_{\tau,W}F) = 0,$$

then

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

Proof. Set

$$A_{F,W,\tau} := \mathcal{R}(C_{\tau,W}F).$$

By eligibility,

$$A_{F,W,\tau} \in D^{[-1,0]}(k\text{-mod}).$$

By Proposition 3.27, the assumption

$$\text{Eval}_{1,W,\tau}(F) = 0$$

implies

$$H^{-1}(A_{F,W,\tau}) = 0.$$

By Corollary 3.25,

$$\text{Ext}^1(A_{F,W,\tau}, k) = 0.$$

Substituting back

$$A_{F,W,\tau} = \mathcal{R}(C_{\tau,W}F)$$

gives the claimed result. $\square$

Remark 3.30 (Discharge theorem). The one-way bridge is a discharge theorem. It states that, in the eligible regime, the repaired topological clause

$$\text{Eval}_{1,W,\tau}(F) = 0$$

is sufficient to discharge the categorical clause

$$\text{Ext}^1 = 0.$$

It does not state that categorical vanishing reconstructs topological vanishing.

C.11. Minimal E_1 -first compatibility

Chapter 3 retains only a minimal operational form of the E_1 -first rule.

Definition 3.31 (Residual degree-1 bar-length functional). Let

$$\text{Eval}_{1,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_1(F))$$

be the repaired windowed collapsed degree-1 persistence profile. Define

$$E_1(F; W, \tau)$$

to be the sum of the lengths of all residual finite degree-1 bars in this profile, together with the convention that the presence of an infinite residual degree-1 bar makes

$$E_1(F; W, \tau) = +\infty.$$

Proposition 3.32 (Minimal E_1 -first implication). *If*

$$E_1(F; W, \tau) = 0,$$

then

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

Consequently, for any admissible representative,

$$\text{PH}_1(C_{\tau,W}F) = 0.$$

Proof. The functional E_1 is the sum of residual degree-1 bar lengths in the repaired windowed collapsed profile, with infinite bars counted as $+\infty$. If $E_1 = 0$, then there are no nonzero residual finite bars and no residual infinite bars in degree 1. Hence

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

If $C_{\tau,W}F$ is an admissible representative, this is equivalent to

$$\text{PH}_1(C_{\tau,W}F) = 0.$$

□

Remark 3.33 (No converse). This appendix uses only the implication

$$E_1(F; W, \tau) = 0 \Rightarrow \text{Eval}_{1,W,\tau}(F) = 0.$$

No converse, local equivalence theorem, or reverse bridge is asserted.

C.12. Gate integration

Proposition 3.34 (Discharge of the Ext^1 -clause of B-Gate⁺). *Let $(F; W, \tau)$ be eligible. If*

$$\text{Eval}_{1,W,\tau}(F) = 0,$$

then the Ext^1 -clause of B-Gate⁺ is satisfied:

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

Proof. This is exactly Theorem 3.29. □

Remark 3.35 (Remaining B-Gate⁺ clauses). The one-way bridge discharges only the categorical Ext^1 -clause. A complete B-Gate⁺ pass still requires:

$$\text{Eval}_{1,W,\tau}(F) = 0, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \Sigma\delta < \text{gap}_{\tau}.$$

If Type IV diagnostics are invoked, the relevant tower comparison artifact must also be declared. Those conditions are handled in Chapters 4 and 6.

C.13. Stability under admissible representative equivalence

Proposition 3.36 (Eligibility is invariant under admissible representative equivalence). *Let*

$$C_{\tau, W}F \sim_{\text{rep}} C'_{\tau, W}F$$

be two admissible representatives of the same repaired windowed collapsed profile. Assume $\mathcal{R}$ is compatible with representative equivalence. Then eligibility of one representative implies eligibility of the other.

Proof. By compatibility of $\mathcal{R}$ with representative equivalence,

$$\mathcal{R}(C_{\tau, W}F) \cong \mathcal{R}(C'_{\tau, W}F)$$

in $D^b(k\text{-mod})$. Thus amplitude $[-1, 0]$, cohomology objects, and the edge-realization condition are preserved up to the declared isomorphism. Hence eligibility is invariant under the admissible change of representative. $\square$

Proposition 3.37 (One-way bridge is representative-independent). *The conclusion of Theorem 3.29 is independent of the admissible representative chosen for $C_{\tau, W}F$, provided the representatives are realization-compatible and equivalent under the declared representative policy.*

Proof. Equivalent admissible representatives induce the same monitored repaired persistence profiles and isomorphic realized objects. Therefore both the hypothesis

$$\text{Eval}_{1, W, \tau}(F) = 0$$

and the conclusion

$$\text{Ext}^1(\mathcal{R}(C_{\tau, W}F), k) = 0$$

are invariant under the admissible representative choice. $\square$

Remark 3.38 (Representative independence is not canonicity). Representative independence does not mean that $C_{\tau, W}F$ is unique or canonical. It means only that the Ext-clause is stable under the declared representative equivalence and realization policy.

C.14. Summary of eligible realization and Ext-edge

Theorem 3.39 (Appendix C eligible-realization package). *Within the eligible realization regime, the following package is available.*

(i) *The repaired windowed collapsed degree- i profile is*

$$\text{Eval}_{i, W, \tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

(ii) *An admissible representative $C_{\tau, W}F$ realizes this profile in the monitored degrees.*

(iii) *The realized object is*

$$A_{F, W, \tau} = \mathcal{R}(C_{\tau, W}F).$$

(iv) *Eligibility requires*

$$A_{F, W, \tau} \in D^{[-1, 0]}(k\text{-mod})$$

together with the edge-realization condition

$$H^{-1}(A_{F, W, \tau}) \cong \mathcal{H}_1^{\tau, W}(F).$$

(v) Over a field, every bounded derived object splits as the direct sum of its cohomology objects.

(vi) For

$$A \in D^{[-1,0]}(k\text{-mod}),$$

one has

$$\text{Ext}^1(A, k) \cong \text{Hom}_k(H^{-1}(A), k).$$

(vii) If

$$H^{-1}(A) = 0,$$

then

$$\text{Ext}^1(A, k) = 0.$$

(viii) If

$$\text{Eval}_{1,W,\tau}(F) = 0,$$

equivalently, for an admissible representative,

$$\text{PH}_1(C_{\tau,W}F) = 0,$$

then

$$H^{-1}(A_{F,W,\tau}) = 0.$$

(ix) Consequently,

$$\text{Eval}_{1,W,\tau}(F) = 0 \implies \text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

(x) All of the above are invariant under admissible realization-compatible representative choices.

Proof. Items (i)–(iii) are Definitions 3.6, 3.7, and 3.13. Item (iv) is Definition 3.14 together with Definition 3.17. Item (v) is Lemma 3.21. Item (vi) is Theorem 3.24. Item (vii) is Corollary 3.25. Item (viii) is Proposition 3.27. Item (ix) is Theorem 3.29. Item (x) follows from Propositions 3.36 and 3.37. $\square$

C.15. Explicit exclusions

The following are not asserted in Appendix C.

- No implication

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0 \implies \text{Eval}_{1,W,\tau}(F) = 0$$

is asserted.

- No implication

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0 \implies \text{PH}_1(C_{\tau,W}F) = 0$$

is asserted.

- No global equivalence

$$\text{PH}_1 \Leftrightarrow \text{Ext}^1$$

is asserted.

- No CCC-dependent reverse principle is introduced.
- No spectral-gap-dependent reverse principle is introduced.

- No Bridge Program is used.
- No validity map is used.
- No Search-layer report is used.
- No AI-generated confidence or ranking is used to discharge the Ext^1 -clause.
- No problem-specific arithmetic, PDE, geometric, or categorical conclusion is proved.
- No realization outside the eligible regime is licensed to invoke the Ext^1 -edge.
- No global strict filtered lift

$$C_\tau : \text{FiltCh}(\cdot)k \rightarrow \text{FiltCh}(\cdot)k$$

is asserted.

- No identity

$$C_{\tau, W}F = (C_\tau F)|_W$$

is asserted.

- No exactness of positive-threshold collapse is used.
- No Serre localization claim for positive-threshold short bars is used.

C.16. Provenance and status

The derived-category facts in this appendix rely only on the semisimplicity of finite-dimensional vector spaces over a field. The bridge itself depends additionally on the eligible realization regime and the edge-realization condition connecting the repaired windowed collapsed degree-1 persistence edge to H^{-1} of the realized object.

Status labels for Appendix C. *Foundation definitions:* Definitions [3.2](#), [3.3](#), [3.4](#), [3.6](#), [3.7](#), [3.10](#), [3.11](#), [3.13](#), [3.14](#), [3.16](#), [3.17](#), [3.23](#), [3.31](#).

Foundation lemmas/propositions/theorems: Lemmas [3.20](#), [3.21](#), Corollary [3.22](#), Theorem [3.24](#), Corollary [3.25](#), Proposition [3.27](#), Theorem [3.29](#), Proposition [3.32](#), Proposition [3.34](#), Proposition [3.36](#), Proposition [3.37](#), Theorem [3.39](#).

Operational cautions: Remarks [3.1](#), [3.8](#), [3.9](#), [3.12](#), [3.15](#), [3.18](#), [3.19](#), [3.26](#), [3.28](#), [3.30](#), [3.35](#), [3.38](#).

Explicit exclusions: Section C.15.

4 Appendix D: Tower Diagnostics and Type IV Defect Calculus

D.1. Scope and dependence on the repaired Core

This appendix supplies the technical foundation for Chapter 4 after the v18.1 Core repair. Its purpose is to make precise the tower comparison artifacts, defect profiles, generic fiber dimensions, and monitored Type IV diagnostics used in the Core.

All tower diagnostics in this appendix are read after the repaired windowed collapsed evaluation

$$\text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

The central distinction is:

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

means absence of monitored generic / infinite-bar Type IV defect relative to a declared tower comparison artifact. It does *not*, by itself, mean that the tower comparison artifact is an isomorphism on all finite-bar components.

This appendix expands the following items:

- (i) directed towers of filtered or persistence-readable objects;
- (ii) repaired windowed collapsed tower profiles;
- (iii) declared tower comparison artifacts;
- (iv) kernel and cokernel defect profiles;
- (v) generic fiber dimension and infinite-bar multiplicity;
- (vi) the diagnostics $\mu_{i,W,\tau}$ and $u_{i,W,\tau}$;
- (vii) monitored generic Type IV obstruction;
- (viii) finite-bar defects and why they are not measured by μ , u ;
- (ix) pure kernel, pure cokernel, and mixed Type IV modes;
- (x) stable bands;
- (xi) admissibility consequences.

Remark 4.1 (Appendix D does not enlarge the Core). This appendix does not introduce a stronger tower theorem than Chapter 4. It only expands the monitored generic Type IV calculus used there. In particular, it does not:

- (i) restore a global exact collapse functor;
- (ii) assert automatic tower comparison maps;
- (iii) upgrade zero μ , u into full comparison-artifact isomorphism;
- (iv) interpret Type IV geometrically, arithmetically, analytically, or categorically without an interface theorem.

D.2. Directed towers in the filtered setting

Definition 4.2 (Directed tower). A *directed tower* in the declared filtered regime is a sequence

$$F_0 \longrightarrow F_1 \longrightarrow F_2 \longrightarrow \cdots$$

of filtered chain complexes and filtration-preserving morphisms. We write this tower as

$$F_\bullet = \{F_n\}_{n \in \mathbb{N}}.$$

Definition 4.3 (Declared apex object). Let $F_\bullet$ be a directed tower. A filtered object F_∞ is called a *declared apex object* if it is supplied as the object against which the repaired collapsed tower profile is compared. It may be an objectwise filtered colimit, a stabilized representative, a completed object, or an externally supplied limiting object, provided its monitored windowed collapsed profiles are constructible.

Remark 4.4 (Apex is declared, not automatic). The notation F_∞ does not assert that an objectwise filtered colimit exists in the finite-type constructible regime. It denotes the declared apex or limiting object for the tower comparison artifact. If the relevant constructibility or representative data are missing, F_∞ is not a Core-readable apex.

Definition 4.5 (Monitored degrees). The *monitored degrees* of a tower are the finitely many homological degrees $i \in I_{\text{mon}}$ for which tower diagnostics are declared and computed. All sums defining total diagnostics are taken only over these monitored degrees.

Remark 4.6 (Constructibility caveat). A raw tower limit may leave the finite-type constructible regime. The Core uses tower diagnostics only when the profiles

$$\text{Eval}_{i,W,\tau}(F_n), \quad \text{Eval}_{i,W,\tau}(F_\infty)$$

and the declared comparison artifacts lie in a constructible persistence-profile regime where the relevant kernel and cokernel profiles are defined.

D.3. Repaired windowed collapsed tower profiles

Fix a right-open window $W = [u, u')$, a threshold $\tau > 0$, and a monitored degree i .

Definition 4.7 (Windowed collapsed finite-stage profile). For a tower stage F_n , the repaired windowed collapsed finite-stage profile is

$$\mathcal{P}_{i,W,\tau}(F_n) := \text{Eval}_{i,W,\tau}(F_n) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F_n)).$$

Definition 4.8 (Windowed collapsed tower profile). The *windowed collapsed tower profile* of $F_\bullet$ in degree i is the sequence

$$\mathcal{P}_{i,W,\tau}(F_0), \mathcal{P}_{i,W,\tau}(F_1), \mathcal{P}_{i,W,\tau}(F_2), \dots$$

together with whatever directed comparison data are explicitly declared for these profiles.

Definition 4.9 (Collapsed colimit profile). A *collapsed colimit profile*

$$\text{ColimProfile}_{i,W,\tau}(F_\bullet)$$

is declared data representing the tower-side repaired windowed collapsed profile. It may be obtained from a genuine constructible colimit, from a stabilized profile, or from a proof/implementation artifact, but it is not automatic in the general Core.

Definition 4.10 (Collapsed apex profile). For a declared apex object F_∞ , the collapsed apex profile is

$$\text{ApexProfile}_{i,W,\tau}(F_\infty) := \text{Eval}_{i,W,\tau}(F_\infty) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F_\infty)).$$

Remark 4.11 (No automatic colimit-after-collapse identity). The repaired Core does not assert a default identity

$$\text{ColimProfile}_{i,W,\tau}(F_\bullet) = \varinjlim_n T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F_n)).$$

Such an identity may be used only when a constructible and functorial profile construction has been declared. This caution is necessary because positive-threshold barcode deletion is not a global exact functor in the general Core.

D.4. Tower comparison artifacts

Definition 4.12 (Tower comparison artifact). A *tower comparison artifact* for (i, W, τ) is declared data specifying a comparison

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty).$$

The artifact must include enough information to compute or certify the monitored kernel and cokernel defect profiles used by the Type IV diagnostics.

Remark 4.13 (Comparison artifact, not automatic map). The map

$$\phi_{i,W,\tau}$$

is not automatic in the general Core. It is either:

- (i) explicitly declared as an artifact;
- (ii) constructed in an implementation-specific profile model;
- (iii) supplied canonically by a separately declared exact-admissible regime.

Without such data, Type IV diagnostics are not well-formed.

Definition 4.14 (Exact-admissible tower artifact). A tower comparison artifact is *exact-admissible* if it is supplied by a regime in which:

- (i) threshold deletion is functorial on the declared tower;
- (ii) the relevant collapsed tower-side profile is constructible;
- (iii) the comparison map is canonical in that regime;
- (iv) kernels and cokernels of the comparison lie in the monitored constructible persistence-profile regime.

Remark 4.15 (Order of operations). The Core tower diagnostic order is:

$$F_n \longmapsto \mathbf{P}_i(F_n) \longmapsto W_W \mathbf{P}_i(F_n) \longmapsto T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F_n)) \longmapsto \text{ColimProfile}_{i,W,\tau}(F_\bullet),$$

and this is compared to

$$F_\infty \longmapsto \mathbf{P}_i(F_\infty) \longmapsto W_W \mathbf{P}_i(F_\infty) \longmapsto T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F_\infty)) = \text{ApexProfile}_{i,W,\tau}(F_\infty).$$

Remark 4.16 (Why repaired collapse precedes diagnostics). The purpose of the repaired windowed collapsed evaluation is to remove short-bar artifacts inside the declared window before tower-level defects are diagnosed. Without this evaluation order, transient bars or windowing artifacts could be misread as persistent tower obstructions.

D.5. Defect profiles

Definition 4.17 (Kernel and cokernel defect profiles). Given a tower comparison artifact

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty),$$

define the monitored defect profiles by

$$\text{Defect}_{i,W,\tau}^{\ker} := \ker(\phi_{i,W,\tau}), \quad \text{Defect}_{i,W,\tau}^{\text{coker}} := \text{coker}(\phi_{i,W,\tau}),$$

whenever the declared profile regime supports these kernel and cokernel computations.

Remark 4.18 (Kernel and cokernel require a profile regime). The expressions

$$\ker(\phi_{i,W,\tau}), \quad \text{coker}(\phi_{i,W,\tau})$$

are meaningful only inside a declared persistence-profile regime where kernels and cokernels are defined and constructible. They are not inferred merely from the existence of a tower of filtered objects.

Remark 4.19 (Interpretation of kernel and cokernel defects). The kernel defect records monitored persistence classes seen in the collapsed tower-side profile that disappear under comparison with the collapsed apex profile. The cokernel defect records monitored persistence classes present in the collapsed apex profile but not supplied by the collapsed tower-side profile. Both are repaired after-collapse defects.

Definition 4.20 (Full defect profile). The *full defect profile* in degree i , window W , and threshold τ is the pair

$$\text{Defect}_{i,W,\tau} := (\text{Defect}_{i,W,\tau}^{\ker}, \text{Defect}_{i,W,\tau}^{\text{coker}}).$$

D.6. Generic fiber dimension

Definition 4.21 (Generic fiber dimension). Let $M \in \text{Pers}_k^{\text{cons}}$. The *generic fiber dimension* of M is

$$\text{gdim}(M) := \lim_{t \rightarrow +\infty} \dim_k M(t),$$

whenever this limit is taken inside the constructible monitored finite-rank regime.

Proposition 4.22 (Existence of generic fiber dimension). *For $M \in \text{Pers}_k^{\text{cons}}$ with locally finite barcode and finite monitored generic rank, the quantity*

$$\text{gdim}(M)$$

exists and equals the eventual dimension of $M(t)$ for sufficiently large t .

Proof. By constructibility and local finiteness, the barcode of M has only finitely many relevant changes in every bounded interval. The eventual fiber dimension is determined by the bars that remain nonzero arbitrarily far to the right. Under finite monitored generic rank, only finitely many such bars contribute in the monitored degree. Thus $\dim_k M(t)$ stabilizes for sufficiently large t , and the limit exists. $\square$

Proposition 4.23 (Generic dimension and infinite-bar multiplicity). *For $M \in \text{Pers}_k^{\text{cons}}$ in the monitored finite-rank regime,*

$$\text{gdim}(M)$$

equals the multiplicity of bars unbounded to $+\infty$ in the barcode of M .

Proof. Finite bars vanish for t sufficiently large. Bars unbounded to $+\infty$ contribute one copy of k to $M(t)$ for all sufficiently large t . Therefore the eventual fiber dimension is exactly the number of such unbounded bars, counted with multiplicity. $\square$

Remark 4.24 (Endpoint conventions). Endpoint conventions do not affect gdim , because generic fiber dimension is computed in the limit $t \rightarrow +\infty$. Only bars unbounded to $+\infty$ contribute.

D.7. Tower diagnostics μ and u

Definition 4.25 (Degree-wise tower diagnostics). For each monitored degree i , window W , and threshold τ , define

$$\mu_{i,W,\tau} := \text{gdim}(\text{Defect}_{i,W,\tau}^{\ker}), \quad u_{i,W,\tau} := \text{gdim}(\text{Defect}_{i,W,\tau}^{\text{coker}}).$$

Definition 4.26 (Total collapse diagnostics). Let I_{mon} be the finite set of monitored degrees. Define

$$\mu_{\text{Collapse}} := \sum_{i \in I_{\text{mon}}} \mu_{i,W,\tau}, \quad u_{\text{Collapse}} := \sum_{i \in I_{\text{mon}}} u_{i,W,\tau}.$$

Remark 4.27 (Meaning of μ). The invariant $\mu_{i,W,\tau}$ measures monitored generic kernel defect: persistent generic information present in the collapsed tower-side profile but killed under comparison with the collapsed apex profile.

Remark 4.28 (Meaning of u). The invariant $u_{i,W,\tau}$ measures monitored generic cokernel defect: persistent generic information appearing in the collapsed apex profile but not supplied by the collapsed tower-side profile.

Proposition 4.29 (Diagnostics count infinite-bar defects). *For each monitored degree i ,*

$$\mu_{i,W,\tau}$$

equals the number of infinite-bar summands in

$$\ker(\phi_{i,W,\tau}),$$

and

$$u_{i,W,\tau}$$

equals the number of infinite-bar summands in

$$\text{coker}(\phi_{i,W,\tau}),$$

within the monitored finite-rank regime.

Proof. This follows immediately from Proposition 4.23 applied to

$$\text{Defect}_{i,W,\tau}^{\ker} \quad \text{and} \quad \text{Defect}_{i,W,\tau}^{\text{coker}}.$$

□

D.8. Monitored generic Type IV obstruction

Definition 4.30 (Monitored generic Type IV obstruction). A declared tower has a *monitored generic Type IV obstruction* on (W, τ) if a tower comparison artifact exists and

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \neq (0, 0).$$

Equivalently, some monitored degree has a nonzero infinite-bar kernel or cokernel defect in the repaired after-collapse tower comparison artifact.

Definition 4.31 (Type IV-free in the monitored generic sense). A declared tower is *Type IV-free in the monitored generic sense* on (W, τ) if a tower comparison artifact exists and

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0).$$

Theorem 4.32 (Criterion for absence of monitored generic Type IV). *Relative to a declared tower comparison artifact, a declared tower is Type IV-free in the monitored generic sense if and only if*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0).$$

Proof. This is exactly Definition 4.31, together with the construction of μ_{Collapse} and u_{Collapse} as generic kernel/cokernel defect counts. $\square$

Remark 4.33 (No full-isomorphism conclusion). Theorem 4.32 does not assert that

$$\phi_{i,W,\tau}$$

is an isomorphism. It asserts only that the infinite-bar part of its kernel and cokernel vanishes in the monitored degrees. Finite-bar defects may still remain unless an additional no-finite-defect condition is imposed.

D.9. Finite-bar defects

Definition 4.34 (Finite-bar defect). A *finite-bar defect* of $\phi_{i,W,\tau}$ is a finite interval summand appearing in either

$$\ker(\phi_{i,W,\tau}) \quad \text{or} \quad \text{coker}(\phi_{i,W,\tau}).$$

Definition 4.35 (No-finite-defect condition). The tower comparison artifact $\phi_{i,W,\tau}$ satisfies the *no-finite-defect condition* if both

$$\ker(\phi_{i,W,\tau}) \quad \text{and} \quad \text{coker}(\phi_{i,W,\tau})$$

have no finite-bar summands.

Proposition 4.36 (Zero diagnostics plus no finite defects imply isomorphism). *If*

$$(\mu_{i,W,\tau}, u_{i,W,\tau}) = (0, 0)$$

and $\phi_{i,W,\tau}$ satisfies the no-finite-defect condition, then

$$\phi_{i,W,\tau}$$

is an isomorphism in the declared constructible persistence-profile regime.

Proof. The kernel and cokernel of $\phi_{i,W,\tau}$ have no infinite-bar summands because

$$(\mu_{i,W,\tau}, u_{i,W,\tau}) = (0, 0).$$

By the no-finite-defect condition, they also have no finite-bar summands. Hence both kernel and cokernel vanish. In the declared abelian constructible persistence-profile regime, $\phi_{i,W,\tau}$ is therefore an isomorphism. $\square$

Remark 4.37 (Why this condition is separated). The Core Type IV diagnostic intentionally monitors the generic / infinite-bar part. Requiring absence of all finite-bar defects is stronger and belongs to an additional verification layer. This separation prevents the Core from silently overclaiming comparison-artifact isomorphism.

D.10. Modes of Type IV

Definition 4.38 (Pure kernel Type IV). A tower has *pure kernel Type IV* on (W, τ) if

$$\mu_{\text{Collapse}} > 0 \quad \text{and} \quad u_{\text{Collapse}} = 0.$$

Definition 4.39 (Pure cokernel Type IV). A tower has *pure cokernel Type IV* on (W, τ) if

$$\mu_{\text{Collapse}} = 0 \quad \text{and} \quad u_{\text{Collapse}} > 0.$$

Definition 4.40 (Mixed Type IV). A tower has *mixed Type IV* on (W, τ) if

$$\mu_{\text{Collapse}} > 0 \quad \text{and} \quad u_{\text{Collapse}} > 0.$$

Remark 4.41 (Interpretation of the three modes). Pure kernel Type IV indicates generic information lost at the apex comparison. Pure cokernel Type IV indicates generic information born at the apex profile. Mixed Type IV indicates both phenomena simultaneously. These descriptions are Core-level diagnostic interpretations only. Any geometric, arithmetic, analytic, or categorical interpretation belongs to an interface appendix.

D.11. Finite admissibility and limit failure

Definition 4.42 (Finite-level collapse admissibility). A finite stage F_n is *finite-level collapse-admissible* on (W, τ) if the local structural conditions

$$\text{Eval}_{1,W,\tau}(F_n) = 0,$$

and, when eligible and invoked,

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F_n), k) = 0$$

hold, and the monitored finite-level diagnostics declared for that stage vanish.

Proposition 4.43 (Finite admissibility does not imply monitored limit admissibility). *There may exist a directed tower $F_\bullet$, a declared apex F_∞ , a window W , a threshold τ , and a tower comparison artifact such that every finite stage is finite-level collapse-admissible, while the declared tower comparison artifact has*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \text{ eq}(0, 0).$$

Proof sketch. Consider a profile tower whose finite stages have no residual degree-1 bars after repaired windowed collapse, while the collapsed apex profile contains a persistent generic class unbounded to $+\infty$ that is not supplied by the collapsed tower-side profile. At each finite stage, the repaired PH_1 - and eligible Ext^1 -checks vanish. However, the comparison artifact

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \rightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty)$$

has nonzero generic cokernel. Thus

$$u_{\text{Collapse}} > 0.$$

This realizes pure cokernel Type IV. Analogous constructions can realize pure kernel or mixed modes by reversing or combining the relevant generic defects. $\square$

Remark 4.44 (Why Type IV is invisible). Type IV is invisible to finite-level admissibility because every finite stage can pass the ordinary structural checks. The obstruction appears only when the collapsed tower-side profile is compared with the collapsed apex profile through a declared tower comparison artifact.

D.12. Stable bands

Definition 4.45 (Degreewise stable band). Fix a monitored degree i , a window W , and a declared tower comparison policy. A contiguous threshold interval

$$B \subset (0, \infty)$$

is called a *degreewise stable band* for i if, for every $\tau \in B$, the tower comparison artifact exists and

$$(\mu_{i,W,\tau}, u_{i,W,\tau}) = (0, 0).$$

Definition 4.46 (Global stable band). A contiguous threshold interval

$$B \subset (0, \infty)$$

is called a *global stable band* if, for every $\tau \in B$, the tower comparison artifacts exist for all monitored degrees and

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0).$$

Proposition 4.47 (Degreewise stable bands imply no degreewise Type IV). *If B is a degreewise stable band for i , then no monitored generic Type IV obstruction occurs in degree i for any $\tau \in B$.*

Proof. This follows directly from Definition 4.45. □

Proposition 4.48 (Global stable bands imply no monitored Type IV). *If B is a global stable band, then the declared tower is Type IV-free in the monitored generic sense for every $\tau \in B$.*

Proof. This is exactly the definition of global stable band combined with Definition 4.31. □

Remark 4.49 (Stable band does not mean full isomorphism band). A stable band controls the monitored generic tower diagnostics. It does not imply that all comparison artifacts are isomorphisms on finite-bar components. A full-isomorphism band requires the additional no-finite-defect condition throughout the band.

D.13. Full-isomorphism bands as optional stronger structure

Definition 4.50 (Full-isomorphism band). A contiguous threshold interval

$$B \subset (0, \infty)$$

is called a *full-isomorphism band* if, for every monitored degree i , every $\tau \in B$, and the declared window W , the comparison artifact

$$\phi_{i,W,\tau}$$

is an isomorphism in the declared constructible persistence-profile regime.

Proposition 4.51 (Full-isomorphism band implies stable band). *Every full-isomorphism band is a global stable band.*

Proof. If $\phi_{i,W,\tau}$ is an isomorphism, then both its kernel and cokernel vanish. Hence

$$\mu_{i,W,\tau} = 0, \quad u_{i,W,\tau} = 0$$

for every monitored degree i , and therefore

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0).$$

□

Remark 4.52 (Converse requires extra data). The converse is not automatic. A global stable band may still contain finite-bar kernel or cokernel defects. To promote a global stable band to a full-isomorphism band, one must also verify the no-finite-defect condition throughout the band.

D.14. Collapse admissibility and tower diagnostics

Definition 4.53 (Tower-clean collapse admissibility). A repaired windowed object $(F; W, \tau)$ is *tower-clean collapse-admissible* if:

(i) the repaired topological clause vanishes:

$$\text{Eval}_{1,W,\tau}(F) = 0;$$

(ii) if the Ext^1 -clause is invoked, the object is eligible and

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0;$$

(iii) the monitored generic tower diagnostics vanish:

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0);$$

(iv) if the tower diagnostics are invoked, the relevant tower comparison artifact is declared.

Proposition 4.54 (Eligible reduction of tower-clean admissibility). *In the eligible regime, tower-clean collapse admissibility is equivalent to:*

$$\text{Eval}_{1,W,\tau}(F) = 0 \quad \text{and} \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0),$$

together with the required declaration of the tower comparison artifact whenever Type IV diagnostics are invoked.

Proof. By the one-way bridge of Chapter 3 and Appendix C,

$$\text{Eval}_{1,W,\tau}(F) = 0$$

implies

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) = 0.$$

Thus, in the eligible regime, the explicit Ext^1 -clause is discharged by the repaired topological clause. The remaining independent tower condition is exactly the vanishing of

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

The artifact declaration remains required because the diagnostics are not well-formed without it. $\square$

Remark 4.55 (Relation to B-Gate⁺). Tower-clean collapse admissibility is the structural part of B-Gate⁺. The final audited pass also requires the budget condition

$$\Sigma\delta < \text{gap}_{\tau}.$$

D.15. Diagnostic stability under admissible equivalence

Definition 4.56 (Diagnostically equivalent tower artifacts). Two tower comparison artifacts

$$\phi_{i,W,\tau}^F \quad \text{and} \quad \phi_{i,W,\tau}^G$$

are *diagnostically equivalent* at (i, W, τ) if their kernel and cokernel defect profiles are isomorphic in the declared constructible persistence-profile regime.

Proposition 4.57 (Invariance of diagnostics). *If two tower comparison artifacts are diagnostically equivalent at (i, W, τ) , then*

$$\mu_{i,W,\tau}(F) = \mu_{i,W,\tau}(G), \quad u_{i,W,\tau}(F) = u_{i,W,\tau}(G).$$

If this holds for all monitored degrees, then

$$\mu_{\text{Collapse}}(F) = \mu_{\text{Collapse}}(G), \quad u_{\text{Collapse}}(F) = u_{\text{Collapse}}(G).$$

Proof. Diagnostic equivalence identifies the kernel and cokernel defect profiles. Generic fiber dimension is invariant under isomorphism in $\text{Pers}_k^{\text{cons}}$. Therefore all degree-wise and total diagnostics agree. $\square$

Remark 4.58 (Representative independence). This proposition ensures that the diagnostics are properties of the declared tower comparison artifact and its defect profiles, not of arbitrary strict representatives chosen in the filtered category.

D.16. Summary of Type IV defect calculus

Theorem 4.59 (Appendix D Type IV diagnostic package). *Within the repaired constructible windowed collapsed tower regime, the following package is available.*

(i) *Each monitored degree i , window W , and threshold τ uses the repaired profile*

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

(ii) *A tower comparison artifact is declared as*

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}) \rightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty}).$$

(iii) *The kernel and cokernel defect profiles are*

$$\text{Defect}_{i,W,\tau}^{\text{ker}} = \ker(\phi_{i,W,\tau}), \quad \text{Defect}_{i,W,\tau}^{\text{coker}} = \text{coker}(\phi_{i,W,\tau}).$$

(iv) *The diagnostics*

$$\mu_{i,W,\tau}, \quad u_{i,W,\tau}$$

are the generic fiber dimensions of these defect profiles.

(v) *The total diagnostics*

$$\mu_{\text{Collapse}}, \quad u_{\text{Collapse}}$$

are finite sums over monitored degrees.

(vi) *Nonzero*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}})$$

is exactly monitored generic Type IV obstruction relative to the declared comparison artifact.

(vii) *Vanishing*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

means absence of monitored generic Type IV obstruction.

(viii) *Zero diagnostics do not imply full comparison-artifact isomorphism unless finite-bar defects are also excluded.*

(ix) *Pure kernel, pure cokernel, and mixed Type IV are distinguished by whether μ , u , or both are nonzero.*

(x) *Stable bands are threshold intervals on which the monitored generic diagnostics remain zero and the relevant comparison artifacts exist.*

(xi) *Full-isomorphism bands are stronger than stable bands and require absence of both generic and finite-bar defects.*

(xii) *In the eligible regime, the structural content of tower-clean collapse admissibility reduces to*

$$\text{Eval}_{1,W,\tau}(F) = 0 \quad \text{and} \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0),$$

together with the required artifact declaration.

Proof. Items (i)–(v) are the definitions of repaired profiles, comparison artifacts, defect profiles, and diagnostics. Items (vi)–(vii) follow from the definition of monitored generic Type IV. Item (viii) follows from the distinction between generic infinite-bar defects and finite-bar defects. Items (ix)–(xi) follow from the definitions of Type IV modes, stable bands, and full-isomorphism bands. Item (xii) follows from the one-way bridge and Proposition 4.54. $\square$

D.17. Explicit exclusions

The following are not asserted in Appendix D.

- No automatic comparison map

$$\lim_{\xrightarrow{n}} \text{Eval}_{i,W,\tau}(F_n) \rightarrow \text{Eval}_{i,W,\tau}(F_\infty)$$

is asserted without a declared tower comparison artifact.

- No automatic comparison map

$$\lim_{\xrightarrow{n}} T_\tau^{\text{bar}}(\mathbf{P}_i(F_n)) \rightarrow T_\tau^{\text{bar}}(\mathbf{P}_i(F_\infty))$$

is asserted in the general Core.

- No global exactness of positive-threshold collapse is used.
- No positive-threshold Serre localization is used.
- No global strict filtered lift C_τ is used.
- Zero μ , u is not asserted to imply full comparison-artifact isomorphism.
- Stable band is not asserted to imply full-isomorphism band.

- Finite-bar defects are not measured by μ , u unless explicitly converted into generic data by an additional policy.
- No local PH–Ext reverse bridge is introduced.
- No Search-layer Lifter, Hunter, Mapper, validity map, or potential Φ is used.
- No geometric, arithmetic, PDE, or categorical interpretation of Type IV is asserted.
- No problem-specific consequence is proved from nonzero Type IV diagnostics.
- No claim is made outside the repaired constructible windowed collapsed tower regime.

D.18. Provenance and status

The material in this appendix is built from:

- (i) the constructible persistence and barcode-level threshold collapse discipline of Appendix A;
- (ii) the admissible representative discipline of Appendix B;
- (iii) the eligible one-way bridge of Appendix C;
- (iv) the tower diagnostic definitions summarized in Chapter 4.

Its distinctive role is to make Type IV a precise monitored generic tower obstruction relative to a declared tower comparison artifact, rather than an informal limit pathology.

Status labels for Appendix D. *Foundation definitions:* Definitions [4.2](#), [4.3](#), [4.5](#), [4.7](#), [4.8](#), [4.9](#), [4.10](#), [4.12](#), [4.14](#), [4.17](#), [4.20](#), [4.21](#), [4.25](#), [4.26](#), [4.30](#), [4.31](#), [4.34](#), [4.35](#), [4.38](#), [4.39](#), [4.40](#), [4.42](#), [4.45](#), [4.46](#), [4.50](#), [4.53](#), [4.56](#).

Foundation propositions/theorems: Propositions [4.22](#), [4.23](#), [4.29](#), [4.36](#), [4.43](#), [4.47](#), [4.48](#), [4.51](#), [4.54](#), [4.57](#), Theorems [4.32](#), [4.59](#).

Operational cautions: Remarks [4.1](#), [4.4](#), [4.6](#), [4.11](#), [4.13](#), [4.15](#), [4.18](#), [4.33](#), [4.37](#), [4.44](#), [4.49](#), [4.52](#), [4.55](#), [4.58](#).

Explicit exclusions: Section D.17.

5 Appendix E: Window, Overlap, Restart, and Summability Details

E.1. Scope and dependence on the repaired Core

This appendix supplies the technical foundation for Chapter 5 after the v18.1 Core repair. Its purpose is to make precise the window logic and local-to-global certificate-record calculus used in the Core.

The central distinctions are the following:

- (i) MECE windowings are used for sequential restart and summability.
- (ii) Overlap-admitting covers are used for compatibility checks and gluing.
- (iii) Local-to-global gluing in the Core is gluing of certificate records, not unrestricted object-level sheaf gluing.

(iv) All windowed persistence data are evaluated by the repaired fixed order

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

(v) Overlap compatibility is evaluated directly on the common overlap unless a separately declared window-adapted commutation condition justifies a different comparison.

This appendix expands the following items:

- (i) right-open windows and endpoint discipline;
- (ii) MECE partitions;
- (iii) overlap-admitting covers;
- (iv) window-restricted certificate records;
- (v) Overlap Gate compatibility;
- (vi) finite Čech depth;
- (vii) certificate-record gluing;
- (viii) Restart lemma;
- (ix) summability;
- (x) countable definable-cover gluing;
- (xi) explicit non-upgrade to object-level gluing.

Remark 5.1 (Appendix E does not enlarge the Core). This appendix does not introduce new Core gates or new problem-specific consequences. It only expands the window, overlap, restart, and summability machinery already used in Chapter 5. No Search-layer map construction, no AI-agent procedure, no unrestricted sheaf-theoretic gluing theorem, and no object-level descent theorem is introduced here.

E.2. Right-open windows

Definition 5.2 (Right-open window). A *right-open window* is an interval

$$W = [u, u') \subset \mathbb{R}$$

with $u < u'$. The left endpoint is included and the right endpoint is excluded.

Definition 5.3 (Declared window range). A *declared window range* is an interval

$$\Omega \subseteq \mathbb{R}$$

on which windowed Core certification is performed. All windows used in a run must be explicitly declared as subintervals of Ω .

Remark 5.4 (Why right-open windows). Right-open windows avoid double-counting at adjacent endpoints. If

$$W_k = [u_k, u_{k+1}) \quad \text{and} \quad W_{k+1} = [u_{k+1}, u_{k+2}),$$

then their intersection is empty under the fixed endpoint convention. Thus each parameter value belongs to exactly one adjacent MECE window.

Definition 5.5 (Window restriction). For a persistence module $M \in \text{Pers}_k^{\text{cons}}$, the window restriction

$$W_W M$$

is the persistence module obtained by restricting M to W , with extension by zero outside W whenever a globally indexed persistence object is required. At the barcode level, W_W clips interval summands to the window and discards empty intersections.

Definition 5.6 (Core window-evaluation order). For a filtered object F , monitored degree i , threshold τ , and window W , the Core windowed collapsed persistence profile is

$$\text{Eval}_{i,W,\tau}(F) := T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

This order is fixed unless a τ -window-adapted or exact-admissible commutation condition is explicitly declared.

Remark 5.7 (No unconditional crop-collapse commutation). In general,

$$W_W T_\tau^{\text{bar}} \not\cong T_\tau^{\text{bar}} W_W.$$

Window restriction can shorten bars. Therefore the Core uses

$$T_\tau^{\text{bar}} W_W$$

as the default windowed evaluation order.

E.3. MECE windowings

Definition 5.8 (MECE windowing). A family of right-open windows

$$\mathcal{W}_{\text{MECE}} = \{W_k\}_{k \in K}, \quad W_k = [u_k, u_{k+1}),$$

is called a *MECE windowing* of a declared range Ω if:

(i) the windows are mutually exclusive:

$$W_k \cap W_\ell = \emptyset \quad (k \neq \ell);$$

(ii) they are collectively exhaustive:

$$\Omega = \bigcup_{k \in K} W_k;$$

(iii) the index order is compatible with the order of the underlying parameter line.

Definition 5.9 (MECE certificate sequence). A *MECE certificate sequence* is a family of window-restricted certificate records

$$\{C_k\}_{k \in K}$$

indexed by a MECE windowing $\{W_k\}_{k \in K}$, where each C_k certifies the declared object on W_k .

Remark 5.10 (MECE windowing is not overlap gluing). A MECE windowing is used for sequential restart and summability. Since the windows are disjoint under the right-open convention, there is no ordinary overlap on which to apply the Overlap Gate. Overlap Gate belongs to overlap-admitting covers, not to strictly MECE partitions.

E.4. Overlap-admitting covers

Definition 5.11 (Overlap-admitting cover). A family of right-open windows

$$\mathcal{U} = \{U_\alpha\}_{\alpha \in A}$$

is called an *overlap-admitting cover* of Ω if

$$\Omega \subseteq \bigcup_{\alpha \in A} U_\alpha$$

and nonempty overlaps

$$U_{\alpha_0} \cap \dots \cap U_{\alpha_p}$$

are allowed and explicitly declared.

Definition 5.12 (Overlap stratum). For indices

$$\alpha_0, \dots, \alpha_p \in A,$$

the associated *overlap stratum* is

$$U_{\alpha_0 \dots \alpha_p} := U_{\alpha_0} \cap \dots \cap U_{\alpha_p}.$$

It is used only when nonempty.

Definition 5.13 (Declared overlap structure). A *declared overlap structure* consists of:

- (i) the overlap-admitting cover $\mathcal{U}$;
- (ii) the nonempty overlap strata to be checked;
- (iii) the overlap budgets attached to those strata;
- (iv) the monitored degrees and diagnostics used on those strata;
- (v) tower comparison artifacts if Type IV diagnostics are invoked on overlaps.

Remark 5.14 (Why overlap data must be declared). Overlap compatibility is part of the audit record. If the overlap structure is not declared, then one cannot reconstruct which local certificate records were compared, which budgets were used, which diagnostics were checked, or which tower comparison artifacts supported Type IV cleanliness.

E.5. Certificate records

Definition 5.15 (Window-restricted certificate record). A *window-restricted certificate record* on a window W at threshold τ consists of:

- (i) the declared object F ;
- (ii) the window W ;
- (iii) the threshold τ ;
- (iv) the monitored degrees i ;

(v) the repaired windowed collapsed persistence profiles

$$\text{Eval}_{i,W,\tau}(F);$$

(vi) the local B-Gate⁺ verdict or structural-pass verdict;

(vii) the local monitored generic Type IV diagnostics;

(viii) the relevant tower comparison artifacts, if Type IV diagnostics are invoked;

(ix) admissible representative data $C_{\tau,W}F$, if filtered or derived clauses are invoked;

(x) the local budget value $\Sigma\delta_W$;

(xi) the local safety margin gap_τ ;

(xii) artifact references sufficient to reconstruct the above data.

Definition 5.16 (Globally coherent certificate record). A family of local certificate records determines a *globally coherent certificate record* if:

- (i) every local record is admissible on its declared window;
- (ii) all declared overlap checks pass, when overlap gluing is used;
- (iii) all restart and summability conditions pass, when MECE restart is used;
- (iv) the cumulative budget semantics are declared;
- (v) every invoked Type IV diagnostic has a declared tower comparison artifact;
- (vi) the resulting global record satisfies the Minimal Manifest Axiom.

Remark 5.17 (Certificate record vs. proof object). A certificate record is the manifest-level representation of a certified proof object. The Core local-to-global logic assembles certificate records. It does not automatically construct a new underlying mathematical object by sheaf-theoretic or categorical gluing.

E.6. Overlap budgets and overlap data

Definition 5.18 (Overlap budget). For a nonempty overlap

$$U_{\alpha\beta} := U_\alpha \cap U_\beta,$$

the *overlap budget*

$$\Sigma\delta_{\alpha\beta}$$

is the declared budget assigned to the comparison of certificate records on $U_{\alpha\beta}$.

Definition 5.19 (Preferred repaired overlap profile). For a monitored degree i , object F , overlap $U_{\alpha\beta}$, and threshold τ , the preferred repaired overlap profile is

$$O_{\alpha\beta,i}(F) := \text{Eval}_{i,U_{\alpha\beta},\tau}(F) = T_\tau^{\text{bar}}(W_{U_{\alpha\beta}}\mathbf{P}_i(F)).$$

Definition 5.20 (Overlap comparison data). Let local records on U_α and U_β be represented by objects F_α and F_β . The default overlap comparison data are the repaired overlap profiles

$$O_{\alpha\beta,i}(F_\alpha) = \text{Eval}_{i,U_{\alpha\beta},\tau}(F_\alpha),$$

and

$$O_{\beta\alpha,i}(F_\beta) = \text{Eval}_{i,U_{\alpha\beta},\tau}(F_\beta).$$

These are evaluated directly on the common overlap $U_{\alpha\beta}$.

Remark 5.21 (Preferred overlap convention). To avoid hidden crop-collapse commutation assumptions, the preferred Core convention is to evaluate overlap records directly on the overlap:

$$\text{Eval}_{i,U_{\alpha\beta},\tau}(-).$$

The Core does not, by default, compare

$$W_{U_{\alpha\beta}} T_\tau^{\text{bar}}(W_{U_\alpha} \mathbf{P}_i(F))$$

with

$$T_\tau^{\text{bar}}(W_{U_{\alpha\beta}} \mathbf{P}_i(F)).$$

Such a comparison requires a separately declared window-adapted or exact-admissible commutation condition.

E.7. Overlap Gate

Definition 5.22 (Overlap Gate). Let U_α and U_β be windows in an overlap-admitting cover, and let

$$U_{\alpha\beta} := U_\alpha \cap U_\beta$$

be nonempty. The *Overlap Gate* passes on $U_{\alpha\beta}$ if the following hold for all monitored degrees.

(i) **Declared overlap data exist.** The repaired overlap profiles

$$\text{Eval}_{i,U_{\alpha\beta},\tau}(F_\alpha), \quad \text{Eval}_{i,U_{\alpha\beta},\tau}(F_\beta)$$

are defined in the declared constructible persistence-profile regime.

(ii) **Overlap discrepancy is controlled.** The two repaired overlap profiles agree up to the declared overlap budget:

$$d_{\text{int}}\left(\text{Eval}_{i,U_{\alpha\beta},\tau}(F_\alpha), \text{Eval}_{i,U_{\alpha\beta},\tau}(F_\beta)\right) \leq \Sigma\delta_{\alpha\beta}.$$

(iii) **Overlap budget is below margin.** The strict inequality

$$\Sigma\delta_{\alpha\beta} < \text{gap}_\tau$$

holds.

(iv) **No monitored generic Type IV appears on the overlap.** The overlap diagnostics satisfy

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

on $U_{\alpha\beta}$ for the declared monitored degrees.

- (v) **Artifact requirement.** If Type IV diagnostics are invoked on the overlap, the relevant tower comparison artifact on $U_{\alpha\beta}$ is declared and auditable.

Remark 5.23 (Overlap Gate is a compatibility test). The Overlap Gate does not create a local certificate. It only checks that two already declared local certificate records are compatible on their common domain within the declared budget and without monitored generic Type IV obstruction.

Proposition 5.24 (Overlap Gate preserves audit readability). *If the Overlap Gate passes on every declared pairwise overlap and the overlap budgets, diagnostics, and required artifacts are recorded, then the pairwise overlap layer of the certificate family is audit-readable.*

Proof. The Overlap Gate records the repaired overlap profiles, the discrepancy bound, the budget comparison, the monitored diagnostics, and any required tower comparison artifacts. These are exactly the data needed to reconstruct the pairwise compatibility checks. $\square$

E.8. Finite Čech depth

Definition 5.25 (Čech nerve). For an overlap-admitting cover

$$\mathcal{U} = \{U_\alpha\}_{\alpha \in A},$$

the Čech nerve $N(\mathcal{U})$ is the simplicial complex whose p -simplices correspond to nonempty intersections

$$U_{\alpha_0} \cap \cdots \cap U_{\alpha_p} \neq \emptyset.$$

Definition 5.26 (Finite Čech depth). The cover $\mathcal{U}$ has *finite Čech depth* K if there are no nonempty intersections of order $K + 1$. Equivalently,

$$\dim N(\mathcal{U}) \leq K - 1.$$

Proposition 5.27 (Finite overlap reduction). *If an overlap-admitting cover has finite Čech depth K , then local-to-global compatibility checks require only overlap strata of order at most K .*

Proof. By definition, all intersections of order $K + 1$ or higher are empty. Thus the Čech nerve has no simplices in dimension K or higher. Therefore all compatibility data live in overlap strata of order at most K . $\square$

Remark 5.28 (Why finite depth is a Core condition). Finite Čech depth keeps the overlap verification auditable. Without a finite-depth bound, a local-to-global certificate record could require uncontrolled higher-overlap data.

E.9. Certificate-record gluing

Definition 5.29 (Compatible local certificate family). Let

$$\{C_\alpha\}_{\alpha \in A}$$

be window-restricted certificate records indexed by an overlap-admitting cover $\mathcal{U} = \{U_\alpha\}$. The family is *compatible* if:

- (i) each C_α is locally admissible;
- (ii) the Overlap Gate passes on every declared nonempty pairwise overlap;

- (iii) all higher-overlap data required by the declared finite Čech-depth policy are recorded;
- (iv) all overlap budgets are included in the ledger;
- (v) all Type IV diagnostics invoked on overlaps have declared tower comparison artifacts.

Theorem 5.30 (Certificate-record local-to-global principle). *Let*

$$\mathcal{U} = \{U_\alpha\}_{\alpha \in A}$$

be an overlap-admitting cover of finite Čech depth. If

$$\{C_\alpha\}_{\alpha \in A}$$

is a compatible local certificate family, then it determines a globally coherent certificate record on

$$\bigcup_{\alpha \in A} U_\alpha.$$

Proof. Finite Čech depth reduces the overlap compatibility problem to finitely many overlap levels. The Overlap Gate supplies pairwise compatibility of repaired overlap profiles within declared budgets and excludes monitored generic Type IV obstruction on overlaps. The declared higher-overlap data ensure coherence through the finite nerve. The ledger records all overlap budgets and artifact requirements. Thus the local certificate records assemble into a globally coherent certificate record satisfying the Minimal Manifest Axiom. $\square$

Remark 5.31 (No unrestricted object-level gluing). Theorem 5.30 is a theorem about certificate records. It does not assert that the underlying filtered complexes, persistence modules, sheaves, or categorical objects glue as mathematical objects. Any such stronger statement belongs to a technical extension appendix and requires a separate proof.

E.10. Restart along MECE partitions

Definition 5.32 (Sequential continuation step). Let

$$W_k = [u_k, u_{k+1}), \quad W_{k+1} = [u_{k+1}, u_{k+2})$$

be adjacent windows in a MECE partition. A *sequential continuation step* from W_k to W_{k+1} is a declared transition policy specifying how the certified run moves from the certificate record on W_k to the certificate record on W_{k+1} .

Definition 5.33 (Per-window budget). For a MECE window W_k , the per-window budget

$$\Sigma \delta_k$$

is the total declared budget used for the local certification of W_k , including algorithmic, discretization, measurement, representative, comparison-artifact, and declared continuation contributions.

Definition 5.34 (Restart margin). If B-Gate⁺ passes on W_k , the *restart margin* on W_k is

$$m_k := \text{gap}_{\tau_k} - \Sigma \delta_k.$$

A window is restart-eligible only if

$$m_k > 0.$$

Lemma 5.35 (Restart lemma). Assume $B\text{-Gate}^+$ passes on W_k , so that

$$m_k = \text{gap}_{\tau_k} - \Sigma\delta_k > 0.$$

Suppose the transition $W_k \rightarrow W_{k+1}$ is governed by a declared admissible continuation class. Then there exists a constant

$$\kappa_k \in (0, 1]$$

depending only on the declared continuation class and threshold policy such that

$$\text{gap}_{\tau_{k+1}} \geq \kappa_k m_k.$$

Proof. The declared continuation class bounds how much certified margin may degrade during the transition from W_k to W_{k+1} . The constant κ_k records the worst admissible degradation under that policy. Since the continuation is admissible and $m_k > 0$, the next margin is bounded below by a positive fraction of m_k . $\square$

Remark 5.36 (Restart is conditional). Restart is not automatic. It requires a positive margin on the preceding window and a declared admissible continuation class. Without these data, a local pass on W_k does not propagate to W_{k+1} .

E.11. Summability

Definition 5.37 (Summability of window budgets). A MECE certificate sequence satisfies *summability* if

$$\sum_k \Sigma\delta_k < \infty$$

in the declared budget semantics.

Definition 5.38 (Summable restart sequence). A MECE certificate sequence is a *summable restart sequence* if:

- (i) $B\text{-Gate}^+$ passes on every W_k ;
- (ii) the Restart lemma applies at every transition $W_k \rightarrow W_{k+1}$;
- (iii) the budgets satisfy

$$\sum_k \Sigma\delta_k < \infty;$$

- (iv) all representatives and comparison artifacts invoked in the sequence are declared.

Theorem 5.39 (Restart–Summability principle). Let

$$\{W_k\}_{k \in K}$$

be a MECE windowing. If the corresponding certificate sequence is a summable restart sequence, then the local certificate records assemble into a globally coherent certificate record on

$$\bigcup_k W_k.$$

Proof. $B\text{-Gate}^+$ provides local certified records. The Restart lemma ensures that positive margin can be propagated along the MECE sequence under the declared continuation policy. Summability prevents the total budget from diverging. Declared representative and comparison-artifact data keep the local verdicts reconstructible. Therefore the local certificate records remain globally auditable and assemble into a globally coherent certificate record. $\square$

Remark 5.40 (Why summability is needed). Without summability, each local window could pass individually while the accumulated budget becomes unbounded. In that case, the global certificate record would fail to be audit-readable as a finite-budget object.

E.12. Countable definable covers

Definition 5.41 (Definable window). A window $W \subset \mathbb{R}$ is *definable* if it is definable in the declared ambient structure, such as an o-minimal structure or a Denef–Pas language, depending on the interface context.

Definition 5.42 (Countable definable cover). A *countable definable cover* of a declared range Ω is a countable family

$$\mathcal{U} = \{U_n\}_{n \geq 1}$$

such that:

(i) each U_n is a right-open definable window;

(ii)

$$\Omega \subseteq \bigcup_{n \geq 1} U_n;$$

(iii) the cover has finite Čech depth;

(iv) the overlap and pipeline budgets are summable.

Theorem 5.43 (Countable definable-cover gluing). *Let*

$$\mathcal{U} = \{U_n\}_{n \geq 1}$$

be a countable definable cover of finite Čech depth. Assume:

(i) *each U_n carries a window-restricted certificate record;*

(ii) *the Overlap Gate passes on every declared nonempty overlap;*

(iii) *all overlap and pipeline budgets are summable;*

(iv) *all required diagnostics, gate verdicts, representative data, tower comparison artifacts, and artifact references are declared.*

Then the local certificate records assemble into a globally coherent Core certificate record on Ω .

Proof. Finite Čech depth ensures that only finitely many overlap levels are needed. Definability prevents uncontrolled pathological cover behavior in the declared ambient structure. Summability ensures that cumulative budget remains finite. Overlap Gate compatibility ensures that local records agree on overlaps within the declared budget and without monitored generic Type IV obstruction. Declared representatives, comparison artifacts, and artifact references ensure audit reconstruction. Thus the local records assemble into a globally coherent certificate record. $\square$

Remark 5.44 (Definability is not decorative). Definability is an audit condition. It prevents the cover from encoding uncontrolled, non-reconstructible overlap complexity. The Core does not use arbitrary countable covers without definability and finite-depth control.

E.13. Budget aggregation across windows and overlaps

Definition 5.45 (Global window budget). For a MECE certificate sequence, the global window budget is

$$\Sigma\delta_{\text{win}} := \sum_k \Sigma\delta_k.$$

Definition 5.46 (Global overlap budget). For an overlap-admitting cover, the global overlap budget is

$$\Sigma\delta_{\text{ov}} := \sum_{\alpha,\beta} \Sigma\delta_{\alpha\beta},$$

where the sum ranges over declared nonempty overlaps included in the certificate record.

Definition 5.47 (Total gluing budget). The total gluing budget is

$$\Sigma\delta_{\text{glue}} := \Sigma\delta_{\text{win}} \oplus \Sigma\delta_{\text{ov}},$$

with aggregation interpreted in the declared budget semantics.

Proposition 5.48 (Budget auditability under summability). *If both the window budgets and overlap budgets are summable, then*

$$\Sigma\delta_{\text{glue}}$$

is audit-readable as a finite budget object in the declared budget semantics.

Proof. Summability of the window budgets gives finiteness of $\Sigma\delta_{\text{win}}$. Summability of the overlap budgets gives finiteness of $\Sigma\delta_{\text{ov}}$. The declared budget aggregation law then yields a finite total gluing budget. $\square$

Remark 5.49 (Relation to gap_τ). A finite total gluing budget is necessary for audit readability, but the final pass condition still requires comparison with the relevant safety margin:

$$\Sigma\delta_{\text{glue}} < \text{gap}_\tau$$

or the appropriate windowwise version declared by the run.

E.14. Compatibility with monitored Type IV diagnostics

Definition 5.50 (Overlap Type IV cleanliness). An overlap stratum is *Type IV-clean in the monitored generic sense* if its overlap diagnostics satisfy

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

for the declared monitored degrees, relative to the declared overlap tower comparison artifacts when such diagnostics are invoked.

Proposition 5.51 (Overlap cleanliness is required for gluing). *A local certificate family cannot be promoted to a globally coherent certificate record through Overlap Gate gluing unless every declared overlap stratum is Type IV-clean in the monitored generic sense.*

Proof. The Overlap Gate includes the condition

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

on each declared overlap. If this fails on any overlap, the Overlap Gate fails there, and the family is not globally coherent by the gluing criterion. $\square$

Remark 5.52 (Type IV cleanliness is generic). As in Appendix D, Type IV cleanliness means absence of monitored generic / infinite-bar defect. It does not by itself assert full comparison-artifact isomorphism on all finite-bar components.

E.15. Object-level gluing as a non-Core strengthening

Definition 5.53 (Object-level gluing claim). An *object-level gluing claim* is a statement asserting that local mathematical objects, such as filtered complexes, persistence modules, sheaves, derived objects, or categorical objects, glue to a global object.

Proposition 5.54 (Certificate gluing does not imply object gluing). *The existence of a globally coherent certificate record does not, by itself, imply an object-level gluing claim.*

Proof. A certificate record contains audit data, gate verdicts, budgets, diagnostics, representative data, comparison artifacts, and artifact references. It records compatibility of local certification data. It does not automatically include descent data, sheaf conditions, cocycle identities, or categorical gluing hypotheses sufficient to construct a global mathematical object. Therefore object-level gluing requires additional hypotheses. $\square$

Remark 5.55 (Where object-level gluing belongs). Object-level gluing, when needed, belongs to a technical extension appendix. It is not part of the Foundation Core unless explicitly imported and proved under the Core guard-rails.

E.16. Summary of window and gluing foundations

Theorem 5.56 (Appendix E window-gluing package). *Within the repaired Core certificate-record regime, the following package is available.*

(i) *Right-open windows provide endpoint-stable local certification domains.*

(ii) *The Core windowed collapsed profile is evaluated as*

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

(iii) *No unconditional commutation*

$$W_W T_{\tau}^{\text{bar}} \cong T_{\tau}^{\text{bar}} W_W$$

is used.

(iv) *MECE windowings are used for sequential restart and summability.*

(v) *Overlap-admitting covers are used for Overlap Gate gluing.*

(vi) *The Overlap Gate compares repaired profiles directly on the common overlap:*

$$\text{Eval}_{i,U_{\alpha\beta},\tau}(-).$$

(vii) *The Overlap Gate checks repaired overlap compatibility, budget margin, monitored generic Type IV cleanliness, and required artifact declaration.*

(viii) *Finite Čech depth bounds the overlap complexity of gluing.*

(ix) *Compatible local certificate records on a finite-depth overlap cover assemble into a globally coherent certificate record.*

(x) *Restart along a MECE sequence requires positive margin and declared admissible continuation.*

- (xi) *Summability is required to prevent cumulative budget divergence.*
- (xii) *Countable definable covers are admissible only under definability, finite-depth, Overlap Gate, artifact declaration, and summability conditions.*
- (xiii) *Certificate-record gluing does not imply unrestricted object-level gluing.*

Proof. Items (i)–(iii) follow from the window convention and the repaired fixed evaluation order. Items (iv)–(v) follow from the distinction between MECE partitions and overlap-admitting covers. Items (vi)–(vii) follow from the definition of the Overlap Gate. Item (viii) follows from finite Čech depth. Item (ix) is Theorem 5.30. Items (x)–(xi) follow from the Restart lemma and summability definitions. Item (xii) follows from Theorem 5.43. Item (xiii) follows from Proposition 5.54. $\square$

E.17. Explicit exclusions

The following are not asserted in Appendix E.

- No unrestricted object-level sheaf gluing theorem is asserted.
- No filtered-complex gluing theorem is asserted.
- No derived-category gluing theorem is asserted.
- No categorical descent theorem is asserted.
- No arbitrary countable cover is admitted without definability and finite Čech depth.
- No overlap compatibility is accepted without declared budgets and diagnostics.
- No overlap Type IV cleanliness is accepted without declared tower comparison artifacts when Type IV diagnostics are invoked.
- No MECE windowing is treated as an overlap-admitting cover.
- No Overlap Gate is used on disjoint MECE windows unless an explicit auxiliary overlap is declared.
- No summability-free global certificate is asserted.
- No Search-layer map, terrain, validity map, or AI-generated report is used to glue certificates.
- No problem-specific arithmetic, PDE, geometric, or categorical conclusion is proved.
- No global exactness of T_τ^{bar} is assumed.
- No global strict filtered lift C_τ is assumed.
- No identity

$$C_{\tau, W}F = (C_\tau F)|_W$$

is assumed.

E.18. Provenance and status

The material in this appendix is built from:

- (i) the window restriction and barcode-level threshold collapse discipline of Appendix A;
- (ii) the admissible representative discipline of Appendix B;
- (iii) the monitored Type IV diagnostic calculus of Appendix D;
- (iv) the audit and manifest discipline of Chapters 1 and 6.

Its role is to provide a precise Core-level account of how local certificate records become globally coherent certificate records without importing Search or object-level gluing.

Status labels for Appendix E. *Foundation definitions:* Definitions [5.2](#), [5.3](#), [5.5](#), [5.6](#), [5.8](#), [5.9](#), [5.11](#), [5.12](#), [5.13](#), [5.15](#), [5.16](#), [5.18](#), [5.19](#), [5.20](#), [5.22](#), [5.25](#), [5.26](#), [5.29](#), [5.32](#), [5.33](#), [5.34](#), [5.37](#), [5.38](#), [5.41](#), [5.42](#), [5.45](#), [5.46](#), [5.47](#), [5.50](#), [5.53](#).

Foundation lemmas/propositions/theorems: Proposition [5.24](#), Proposition [5.27](#), Theorem [5.30](#), Lemma [5.35](#), Theorem [5.39](#), Theorem [5.43](#), Proposition [5.48](#), Proposition [5.51](#), Proposition [5.54](#), Theorem [5.56](#).

Operational cautions: Remarks [5.1](#), [5.4](#), [5.7](#), [5.10](#), [5.14](#), [5.17](#), [5.21](#), [5.31](#), [5.36](#), [5.40](#), [5.44](#), [5.52](#), [5.55](#).

Explicit exclusions: Section E.17.

6 Appendix G: Minimal Manifest and Proof Object Schema

G.1. Scope and dependence on the repaired Core

This appendix supplies the abstract proof-object and manifest foundation for Chapters 1 and 6 after the v18.1 Core repair. Chapter 1 introduced the Minimal Manifest Axiom. Chapter 6 explained why audit pass/reject requires declared windowed evaluation records, ledger data, gate verdicts, monitored diagnostics, representative data when used, tower comparison artifacts when invoked, and artifact references.

The purpose of this appendix is to specify, at the Core level, the abstract schema of:

- (i) certified proof objects;
- (ii) certificate records;
- (iii) declared windows;
- (iv) threshold and collapse policies;
- (v) repaired windowed evaluation records;
- (vi) admissible representative records;
- (vii) ledger policies;
- (viii) monitored diagnostics;

- (ix) tower comparison artifacts;
- (x) gate verdicts;
- (xi) artifact references;
- (xii) audit-readable reconstruction.

Remark 6.1 (Appendix G is abstract, not executable). This appendix defines the abstract Core schema required for auditability. It does not prescribe a concrete serialization format such as YAML, JSON, CSV, database tables, proof-store layout, or replay protocol. Concrete execution formats belong to the Execution/Reproducibility Companion.

Remark 6.2 (Appendix G does not enlarge the Core). This appendix does not introduce new mathematical theorems beyond the Minimal Manifest and audit discipline already used in the main text. It only specifies what information must be present for a proof object or certificate record to be admissible.

G.2. Proof objects and certificate records

Definition 6.3 (Proof object). A *proof object* is an abstract object intended to support a Core claim by recording the data necessary to reconstruct its certification. A proof object is not merely a mathematical assertion. It is a structured collection of declarations, repaired evaluation records, diagnostics, verdicts, budgets, representative data when used, comparison artifacts when invoked, and references.

Definition 6.4 (Certified proof object). A proof object is called *certified* if it satisfies the Minimal Manifest Axiom and its associated evaluation, gate, diagnostic, artifact, and budget checks. In particular, it must expose enough data for an independent verifier to reconstruct the relevant Core verdict.

Definition 6.5 (Certificate record). A *certificate record* is the manifest-level representation of a certified proof object. It contains the declared data required to evaluate, reproduce, or audit the corresponding proof object.

Remark 6.6 (Proof object vs. certificate record). A proof object is the abstract certified entity. A certificate record is its audit-readable representation. The Core operates on the existence and adequacy of such records, not on any particular file format.

Definition 6.7 (Local certificate record). A *local certificate record* is a certificate record attached to a declared window W , threshold τ , monitored degrees, repaired windowed collapsed profiles, local diagnostics, local ledger data, and local gate verdicts.

Definition 6.8 (Global certificate record). A *global certificate record* is a certificate record assembled from local certificate records by the Core local-to-global logic, restart/summability logic, or other declared certificate-record assembly principles. It must retain references to all local records and all compatibility checks used in its construction.

Remark 6.9 (Global record is not object-level gluing). A global certificate record is not, by itself, an object-level gluing theorem. It records globally coherent certification data. Any assertion that underlying mathematical objects glue requires separate hypotheses and proof.

G.3. Minimal Manifest Axiom

Axiom 6.1 (Minimal Manifest Axiom). *A certified proof object is admissible only if it contains, at minimum, the following declared data.*

- (i) *declared objects;*
- (ii) *declared windows and window structure;*
- (iii) *threshold policy;*
- (iv) *collapse policy;*
- (v) *repaired evaluation order and evaluation records;*
- (vi) *representative policy and representative artifacts, if filtered or derived clauses are invoked;*
- (vii) *eligibility data, if the Ext^1 -clause is invoked;*
- (viii) *ledger policy;*
- (ix) *monitored diagnostics;*
- (x) *tower comparison artifacts, if Type IV diagnostics are invoked;*
- (xi) *gate verdicts;*
- (xii) *artifact references;*
- (xiii) *final status classification.*

Remark 6.10 (Meaning of minimality). The word “minimal” means that these data are necessary for Core auditability. A concrete implementation may store additional metadata, timestamps, software versions, hashes, runtime logs, provenance traces, or proof-assistant outputs. Such additional data may improve reproducibility, but the categories in Axiom 6.1 are the Core-level minimum after the v18.1 repair.

Proposition 6.11 (Necessity of manifest data). *If any required manifest component in Axiom 6.1 is absent, then the corresponding proof object is not audit-readable in the Core sense.*

Proof. Without declared objects and windows, the certification domain is unknown. Without threshold and collapse policies, the repaired windowed collapse operation is not determined. Without evaluation records, the Core profile

$$\text{Eval}_{i,W,\tau}(F)$$

cannot be reconstructed. Without representative and eligibility data, filtered or derived clauses cannot be audited. Without ledger policy, the budget comparison cannot be reconstructed. Without monitored diagnostics and tower comparison artifacts, Type IV checks cannot be verified. Without gate verdicts, pass/reject status is not recorded. Without artifact references, the declared data cannot be traced to their evidential source. Without final status classification, the record cannot be placed in the claim taxonomy. Thus the proof object is not audit-readable. $\square$

G.4. Declared objects and windows

Definition 6.12 (Declared object data). The *declared object data* of a certificate record consist of:

- (i) the object F being certified, or the external object X and its projected object F_X when an interface is used;
- (ii) the ambient regime, such as $\text{FiltCh}(\cdot)k$ or $\text{Pers}_k^{\text{cons}}$;
- (iii) the monitored degrees;
- (iv) the source of the object data;
- (v) artifact references sufficient to reconstruct the declared object at the relevant level.

Definition 6.13 (Declared window data). The *declared window data* of a certificate record consist of:

- (i) the window or windows used for certification;
- (ii) their endpoint convention, normally right-open;
- (iii) whether the family is a MECE windowing or an overlap-admitting cover;
- (iv) the declared range covered by the windows;
- (v) any overlap structure needed for local-to-global assembly.

Definition 6.14 (Window identifier). A *window identifier* is a label assigned to a declared window W , allowing all evaluation records, diagnostics, budgets, gate verdicts, representatives, comparison artifacts, and artifacts associated to W to be referenced unambiguously.

Remark 6.15 (Window declarations and Appendix E). The distinction between MECE windowings and overlap-admitting covers follows Appendix E. MECE windowings support restart and summability. Overlap-admitting covers support Overlap Gate and certificate-record gluing.

Proposition 6.16 (Window data are audit-critical). A *certificate record without declared window data cannot be evaluated by the Core window logic*.

Proof. Core evaluation is windowed. The repaired windowed collapsed persistence profile is read as

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Without W , this profile is not determined. Therefore neither local gate verdicts nor overlap checks can be reconstructed. $\square$

G.5. Threshold and collapse policy

Definition 6.17 (Threshold policy). A *threshold policy* specifies how collapse thresholds are chosen and interpreted. It must include:

- (i) the threshold value τ , or a declared family of thresholds;
- (ii) whether the run uses a fixed threshold or a threshold sweep;
- (iii) the rule for comparing results across thresholds;

- (iv) any declared stable band;
- (v) any window-dependent threshold assignment.

Definition 6.18 (Collapse policy). A *collapse policy* specifies which status of threshold collapse is used in the certificate record. The permitted Core-level declarations include:

$$\text{barcode_level}, \quad \text{tau_split}, \quad \text{exact_admissible}.$$

The default repaired Core policy is

$$\text{barcode_level}.$$

Definition 6.19 (Fixed-threshold certificate). A certificate record is a *fixed-threshold certificate* if all Core gate, diagnostic, and budget checks are evaluated at one declared value $\tau > 0$.

Definition 6.20 (Threshold-sweep certificate). A certificate record is a *threshold-sweep certificate* if it records certification data over a declared family

$$\{\tau_\lambda\}_{\lambda \in \Lambda}$$

or a declared threshold band

$$B \subset (0, \infty).$$

Such a record must specify how pass, fail, stability, or bandwise claims are interpreted.

Remark 6.21 (Threshold policy and repaired collapse). The threshold policy determines the barcode-level deletion rule

$$T_\tau^{\text{bar}}.$$

It does not, by default, determine a global exact functor or Serre localization. Any use of stronger exactness must be declared through an exact-admissible collapse policy.

Proposition 6.22 (Threshold and collapse data are audit-critical). *A certificate record without threshold and collapse policy cannot determine the relevant repaired windowed collapsed profile.*

Proof. The repaired evaluation depends on both W and τ :

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Different thresholds delete different finite bars. Different collapse policies license different levels of functorial or exact use. Therefore the gate verdict, diagnostics, and budget margin cannot be reconstructed without knowing τ and the declared collapse policy. $\square$

G.6. Repaired evaluation records

Definition 6.23 (Evaluation order record). An *evaluation order record* specifies the order

$$\mathbf{P}_i \longrightarrow W_W \mathbf{P}_i \longrightarrow T_\tau^{\text{bar}}(W_W \mathbf{P}_i).$$

Equivalently, it declares that the gate-relevant object is

$$\text{Eval}_{i,W,\tau}(F).$$

Definition 6.24 (Repaired evaluation record). A *repaired evaluation record* is a manifest entry specifying, for each declared monitored degree i , window W , and threshold τ , the profile

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)),$$

together with artifact references sufficient to reconstruct it.

Definition 6.25 (Overlap evaluation record). An *overlap evaluation record* is a repaired evaluation record computed directly on a declared overlap stratum $U_{\alpha\beta}$:

$$\text{Eval}_{i,U_{\alpha\beta},\tau}(F).$$

It is used by the Overlap Gate.

Remark 6.26 (No hidden crop-collapse commutation). A certificate record must not silently replace

$$\text{Eval}_{i,U_{\alpha\beta},\tau}(F)$$

by a restriction of an already-collapsed larger-window profile unless a window-adapted or exact-admissible commutation condition is explicitly declared.

Proposition 6.27 (Evaluation records are audit-critical). A *certificate record without repaired evaluation records cannot reconstruct the topological clause of $B\text{-Gate}^+$* .

Proof. The topological clause is stated at the repaired profile level:

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

Without the evaluation record or the artifacts required to reconstruct it, the verifier cannot check this clause. $\square$

G.7. Representative and eligibility records

Definition 6.28 (Representative policy). A *representative policy* specifies whether the certificate record uses admissible filtered representatives

$$C_{\tau,W}F.$$

If representatives are used, the policy must declare:

- (i) the monitored degrees represented;
- (ii) the representative equivalence policy;
- (iii) the relation

$$\mathbf{P}_i(C_{\tau,W}F) \cong \text{Eval}_{i,W,\tau}(F);$$

- (iv) artifact references for the representative construction;
- (v) realization compatibility, if derived clauses are invoked.

Definition 6.29 (Representative record). A *representative record* is the manifest entry recording an admissible representative

$$C_{\tau,W}F$$

and the data required to verify that it realizes the repaired windowed collapsed profile in the monitored degrees.

Definition 6.30 (Eligibility record). An *eligibility record* is the manifest entry required when the Ext^1 -clause is invoked. It must declare:

- (i) the admissible representative $C_{\tau,WF}$;
- (ii) the realization functor or realization procedure $\mathcal{R}$;
- (iii) the realized object

$$\mathcal{R}(C_{\tau,WF});$$

- (iv) the amplitude condition

$$\mathcal{R}(C_{\tau,WF}) \in D^{[-1,0]}(k\text{-mod});$$

- (v) the edge-realization condition linking the repaired residual degree-1 profile to H^{-1} .

Remark 6.31 (No representative, no derived clause). If the Ext^1 -clause is invoked, representative and eligibility records are mandatory. Without them, the certificate record has no typed route from

$$\text{Eval}_{1,W,\tau}(F)$$

to

$$\text{Ext}^1(\mathcal{R}(C_{\tau,WF}), k).$$

Proposition 6.32 (Representative and eligibility records are audit-critical when invoked). *If a certificate invokes filtered or derived clauses, then missing representative or eligibility records make the certificate non-audit-readable.*

Proof. Filtered and derived clauses are not evaluated directly on raw persistence data. They require an admissible representative $C_{\tau,WF}$, and the Ext^1 -clause requires eligibility. If those records are missing, the verifier cannot reconstruct the filtered or derived component of the gate. $\square$

G.8. Ledger policy

Definition 6.33 (Ledger policy). A *ledger policy* specifies the budget semantics used in a certificate record. It must include:

- (i) the budget value domain, usually a declared commutative quantale;
- (ii) the aggregation law;
- (iii) the budget components being aggregated;
- (iv) the total budget $\Sigma\delta$;
- (v) the safety margin gap_{τ} ;
- (vi) the strict pass condition

$$\Sigma\delta < \text{gap}_{\tau}.$$

Definition 6.34 (Budget component). A *budget component* is a declared contribution to the total budget. Typical examples include:

- (i) algorithmic discrepancy;

- (ii) discretization discrepancy;
- (iii) measurement discrepancy;
- (iv) continuation discrepancy;
- (v) overlap discrepancy;
- (vi) representative discrepancy;
- (vii) comparison-artifact discrepancy;
- (viii) implementation discrepancy.

Definition 6.35 (Total budget record). A *total budget record* is the manifest entry specifying

$$\Sigma\delta$$

together with its decomposition into declared budget components and its aggregation rule.

Definition 6.36 (Safety margin record). A *safety margin record* is the manifest entry specifying

$$\text{gap}_\tau$$

and explaining how it is associated with the declared threshold, window, and gate condition.

Remark 6.37 (Strict inequality). The Core uses the strict budget pass condition

$$\Sigma\delta < \text{gap}_\tau.$$

The boundary case

$$\Sigma\delta = \text{gap}_\tau$$

is not a pass. This convention must be reflected in the ledger policy.

Proposition 6.38 (Ledger data are audit-critical). A *certificate record without ledger policy cannot determine Audit Mode pass/reject*.

Proof. By Chapter 6, audited acceptance requires the budget comparison

$$\Sigma\delta < \text{gap}_\tau.$$

Without ledger semantics, $\Sigma\delta$, and gap_τ , this comparison cannot be reconstructed. □

G.9. Monitored diagnostics and tower comparison artifacts

Definition 6.39 (Diagnostic policy). A *diagnostic policy* specifies which obstruction diagnostics are monitored in a certificate record. At minimum, when tower data are involved, it must specify:

- (i) the monitored degrees;
- (ii) the declared windows and thresholds;
- (iii) the tower comparison artifacts;
- (iv) the kernel and cokernel defect profiles;

(v) the diagnostics

$$\mu_{i,W,\tau}, \quad u_{i,W,\tau};$$

(vi) the total diagnostics

$$\mu_{\text{Collapse}}, \quad u_{\text{Collapse}};$$

(vii) whether the diagnostic verdict is $(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$ or $(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \text{ eq}(0, 0)$.

Definition 6.40 (Tower comparison artifact record). A *tower comparison artifact record* is a manifest entry specifying a declared comparison

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty),$$

together with the data needed to compute or certify its kernel and cokernel defect profiles.

Definition 6.41 ($(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$ record). A $(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$ *record* is a manifest entry asserting

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

for the declared windows, thresholds, monitored degrees, and tower comparison artifacts.

Definition 6.42 ($(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \text{ eq}(0, 0)$ record). A $(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \text{ eq}(0, 0)$ *record* is a manifest entry asserting

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \text{ eq}(0, 0)$$

for the declared windows, thresholds, monitored degrees, and tower comparison artifacts.

Remark 6.43 (Generic Type IV meaning). The diagnostic record follows Appendix D. Thus $(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$ means absence of monitored generic / infinite-bar Type IV defect. It does not automatically assert full comparison-artifact isomorphism on all finite-bar components.

Proposition 6.44 (Diagnostic and comparison-artifact data are audit-critical). A *certificate record without diagnostic data and required tower comparison artifacts cannot determine the Type IV component of B-Gate⁺*.

Proof. B-Gate⁺ requires the diagnostic condition

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0).$$

By Appendix D, these diagnostics are defined relative to declared tower comparison artifacts. Without monitored diagnostic data and the required comparison artifacts, this condition cannot be reconstructed. $\square$

G.10. Gate verdicts

Definition 6.45 (Gate verdict). A *gate verdict* is a declared pass/fail outcome of a Core gate. The primary gate verdict in the Core is the verdict of B-Gate⁺. When overlap gluing is used, the Overlap Gate verdicts must also be recorded.

Definition 6.46 (B-Gate⁺ verdict record). A *B-Gate⁺ verdict record* contains:

(i) the repaired topological gate result:

$$\text{Eval}_{1,W,\tau}(F) = 0 \quad \text{or not};$$

- (ii) if the Ext^1 -clause is invoked, the eligibility record;
- (iii) if invoked and eligible, the categorical gate result:

$$\text{Ext}^1(\mathcal{R}(C_{\tau,WF}), k) = 0 \quad \text{or not};$$

- (iv) the monitored Type IV diagnostic result:

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0) \quad \text{or} \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) \text{ eq } (0, 0);$$

- (v) the tower comparison artifact status, if Type IV diagnostics are invoked;
- (vi) the budget result:

$$\Sigma\delta < \text{gap}_{\tau} \quad \text{or not};$$

- (vii) the final pass/fail status.

Definition 6.47 (Overlap Gate verdict record). *An Overlap Gate verdict record contains:*

- (i) the overlap window or stratum;
- (ii) the compared local certificate records;
- (iii) the repaired overlap evaluation records

$$\text{Eval}_{i,U_{\alpha\beta},\tau}(-);$$

- (iv) the overlap discrepancy value;
- (v) the overlap budget;
- (vi) the overlap diagnostic result;
- (vii) the tower comparison artifact status, if Type IV diagnostics are invoked on the overlap;
- (viii) the final pass/fail status.

Remark 6.48 (Verdicts are not scores). A gate verdict is not a confidence score, ranking, heuristic estimate, or Search-side plausibility measure. It is a declared Core pass/fail outcome computed from the relevant gate conditions.

Proposition 6.49 (Gate verdicts are audit-critical). *A certificate record without gate verdicts cannot determine whether the Core accepted or rejected the proof object.*

Proof. The final audited status is a pass/fail determination. Without the recorded gate verdicts, the certificate record does not state the outcome of the Core verifier. $\square$

G.11. Artifact references

Definition 6.50 (Artifact). An *artifact* is any concrete evidential object used to support or reconstruct a certificate record. Examples include:

- (i) barcode data;
- (ii) persistence module data;
- (iii) filtered-complex data;
- (iv) admissible representative data;
- (v) tower comparison artifacts;
- (vi) diagnostic tables;
- (vii) ledger records;
- (viii) proof traces;
- (ix) formal proof objects;
- (x) computation logs;
- (xi) source files;
- (xii) external references.

Definition 6.51 (Artifact reference). An *artifact reference* is a stable identifier or pointer that allows an artifact to be located, checked, and associated with the relevant certificate record.

Definition 6.52 (Artifact reference policy). An *artifact reference policy* specifies:

- (i) which artifacts support the certificate record;
- (ii) how each artifact is identified;
- (iii) which manifest entries each artifact supports;
- (iv) whether the artifact is required for verification, reproduction, or explanatory context;
- (v) the boundary between Core-required artifacts and companion-level execution artifacts.

Remark 6.53 (Artifact references are not themselves theorems). An artifact reference does not turn an artifact into a theorem. It only records where supporting evidence or reconstructive data may be found. The artifact must still be interpreted through the Core verifier discipline.

Proposition 6.54 (Artifact references are audit-critical). *A certificate record without artifact references cannot be independently reconstructed.*

Proof. The record may state windows, thresholds, evaluation records, budgets, diagnostics, and verdicts, but without artifact references an auditor cannot trace those statements to their supporting data. Thus the record fails the reproducibility side of the Minimal Manifest Axiom. $\square$

G.12. Audit-readable records

Definition 6.55 (Audit-readable certificate record). A certificate record is *audit-readable* if an independent verifier can reconstruct:

- (i) the declared objects;
- (ii) the declared windows and window structure;
- (iii) the threshold and collapse policies;
- (iv) the repaired evaluation records;
- (v) the representative and eligibility records, when invoked;
- (vi) the ledger policy and budget comparison;
- (vii) the monitored diagnostics and tower comparison artifacts;
- (viii) the gate verdicts;
- (ix) the artifact references supporting the above data;
- (x) the final status classification.

Definition 6.56 (Audit reconstruction). An *audit reconstruction* is the process of recomputing or checking the Core verdict from the certificate record and its referenced artifacts. It must reproduce the same pass/fail outcome under the declared policies.

Proposition 6.57 (Minimal Manifest implies audit readability). *If a certificate record satisfies the Minimal Manifest Axiom and its artifact references are sufficient to reconstruct the declared data, then it is audit-readable.*

Proof. The Minimal Manifest Axiom supplies the required categories of data. Sufficient artifact references make those data reconstructible. Therefore an independent verifier can reconstruct the declared objects, windows, threshold and collapse policies, repaired evaluation records, representative and eligibility records when invoked, ledger comparison, diagnostics, tower comparison artifacts, gate verdicts, and final status. $\square$

Proposition 6.58 (Audit readability implies verifier independence). *An audit-readable certificate record can be evaluated by a verifier that did not participate in producing the original record.*

Proof. By definition, audit readability requires that the verifier can reconstruct the relevant Core verdict from declared data and artifact references. No access to the original proposer or Search mechanism is required. $\square$

Remark 6.59 (Relation to Core sovereignty). Audit readability supports Core sovereignty. The verifier checks repaired evaluations, gates, diagnostics, artifacts, and budgets from manifest data. Search-side confidence, proposal history, or AI-agent ranking is not part of the verdict.

G.13. Proposer–verifier separation in the manifest

Definition 6.60 (Proposer-side data). *Proposer-side data* are data generated by candidate-producing mechanisms, including Search algorithms, terrain maps, heuristic rankings, validity maps, bridge candidates, or AI-agent suggestions.

Definition 6.61 (Verifier-side data). *Verifier-side data* are data required by the Core verifier: declared objects, windows, thresholds, collapse policies, repaired evaluation records, representative and eligibility records when invoked, ledgers, diagnostics, tower comparison artifacts, gate verdicts, artifact references, and final status classifications.

Proposition 6.62 (Only verifier-side data determine Core verdicts). *The Core pass/reject verdict is determined only by verifier-side data. Proposer-side data may be recorded for provenance, but they do not alter the verdict.*

Proof. By Chapter 6, Core verdicts are determined only by repaired windowed evaluation records, eligible Ext verdicts when invoked, monitored diagnostics, declared artifacts, and budget comparison. These are verifier-side data. Proposer-side data do not appear in the Core audit rule. $\square$

Remark 6.63 (Recording proposer data). A certificate record may include proposer-side data as optional provenance. However, such data must be clearly marked as non-verdict data. They cannot replace missing verifier-side data.

G.14. Abstract schema of a certificate record

Definition 6.64 (Abstract certificate schema). An abstract certificate record has the following schema:

$$C = \left(\text{Obj, Win, Thr, Collapse, Eval, Rep, Elig, Diag, Cmp, Ledger, Gate, Artifact, Status} \right),$$

where:

- (i) Obj records the declared object F , or external object X and projected object F_X ;
- (ii) Win records window data;
- (iii) Thr records threshold policy;
- (iv) Collapse records collapse policy;
- (v) Eval records repaired windowed collapsed profiles;
- (vi) Rep records admissible representative declarations, if used;
- (vii) Elig records eligibility data, if the Ext¹-clause is invoked;
- (viii) Diag records monitored diagnostics;
- (ix) Cmp records tower comparison artifacts, if invoked;
- (x) Ledger records budget semantics and values;
- (xi) Gate records gate verdicts;
- (xii) Artifact records artifact references;

(xiii) Status records the final pass/fail or non-claim status.

Remark 6.65 (Schema is abstract). The tuple in Definition 6.64 is an abstract schema. It is not a required file format. Concrete fields, filenames, hashes, and serialization syntax belong to the Execution/Reproducibility Companion.

Definition 6.66 (Status field). The Status field of a certificate record must classify the record as one of:

- (i) Core pass;
- (ii) Core reject;
- (iii) incomplete;
- (iv) operational-only;
- (v) interface-level;
- (vi) Search artifact;
- (vii) non-claim;
- (viii) [Spec].

Proposition 6.67 (Schema sufficiency for Core audit). *If an abstract certificate record contains all schema components in Definition 6.64, and each component is populated with sufficient reconstructive data, then the record is sufficient for Core audit.*

Proof. The schema includes object, window, threshold, collapse, evaluation, representative, eligibility, diagnostic, comparison-artifact, ledger, gate, artifact, and status data. These cover all components of the Minimal Manifest Axiom and the audit pass/reject discipline. Thus a verifier can reconstruct the Core verdict. $\square$

G.15. Incomplete and failed records

Definition 6.68 (Incomplete certificate record). A certificate record is *incomplete* if one or more required manifest components are missing or not reconstructible.

Definition 6.69 (Failed certificate record). A certificate record is *failed* if all required manifest components are present but one or more Core evaluation, gate, diagnostic, artifact, or budget conditions fail.

Remark 6.70 (Incomplete vs. failed). Incomplete and failed records are different. An incomplete record cannot be audited. A failed record can be audited and yields a Core reject verdict.

Proposition 6.71 (Failure is preferable to incompleteness). *From the viewpoint of auditability, a failed certificate record is more informative than an incomplete certificate record.*

Proof. A failed record exposes enough data to reconstruct why the Core rejected it. An incomplete record lacks the data needed for reconstruction. Thus failure is auditable, while incompleteness is not. $\square$

G.16. Relation to companions and execution schemas

Definition 6.72 (Execution schema). An *execution schema* is a concrete serialization or runtime format for recording certificate data, such as YAML, JSON, CSV, log files, proof assistant outputs, or database entries.

Remark 6.73 (Appendix G vs. execution companion). Appendix G specifies what must exist at the Core manifest level. The Execution/Reproducibility Companion specifies how such data may be serialized, hashed, stored, replayed, or checked by concrete systems. The former is part of the Foundation Appendices. The latter is an execution companion.

Proposition 6.74 (Concrete execution schemas must refine the abstract schema). *Any concrete execution schema used to support a Core certificate must refine the abstract certificate schema of Definition 6.64.*

Proof. A concrete schema is admissible only if it records the data required by the abstract schema. Otherwise, it would fail the Minimal Manifest Axiom or the audit-readable reconstruction requirement. $\square$

G.17. Summary of manifest foundations

Theorem 6.75 (Appendix G manifest package). *Within the Core audit regime, the following package is available.*

- (i) *A proof object is certified only if it satisfies the Minimal Manifest Axiom.*
- (ii) *A certificate record is the manifest-level representation of a certified proof object.*
- (iii) *The Minimal Manifest requires declared object data, window data, threshold policy, collapse policy, repaired evaluation records, representative and eligibility records when invoked, ledger policy, monitored diagnostics, tower comparison artifacts when invoked, gate verdicts, artifact references, and final status classification.*
- (iv) *Declared windows determine the local domain of certification.*
- (v) *Threshold and collapse policy determine the repaired barcode-level threshold deletion used in*

$$\text{Eval}_{i,w,\tau}(F).$$
- (vi) *Repaired evaluation records determine the topological clause of $B\text{-Gate}^+$.*
- (vii) *Representative and eligibility records determine whether filtered and derived clauses may be invoked.*
- (viii) *Ledger policy determines the budget comparison*

$$\Sigma\delta < \text{gap}_\tau.$$
- (ix) *Monitored diagnostics and tower comparison artifacts determine the Type IV component of $B\text{-Gate}^+$.*
- (x) *Gate verdicts determine the local Core pass/reject status.*
- (xi) *Artifact references make the record reconstructible.*
- (xii) *Audit-readable records support verifier independence.*
- (xiii) *Proposer-side data do not determine Core verdicts.*

(xiv) *Concrete execution schemas must refine the abstract certificate schema, but are not fixed by this appendix.*

Proof. Items (i)–(iii) follow from the Minimal Manifest Axiom. Items (iv)–(xi) follow from the definitions of object, window, threshold, collapse, evaluation, representative, eligibility, ledger, diagnostic, comparison-artifact, gate, and artifact data. Item (xii) follows from audit readability. Item (xiii) follows from Core sovereignty and proposer–verifier separation. Item (xiv) follows from the distinction between the abstract schema and concrete execution schemas. $\square$

G.18. Explicit exclusions

The following are not asserted in Appendix G.

- No concrete YAML, JSON, CSV, database, or proof-store format is fixed.
- No execution runtime is specified.
- No hash policy is fixed.
- No file layout is fixed.
- No Search artifact is promoted to proof-object status.
- No AI confidence value is treated as a gate verdict.
- No validity map is treated as a theorem.
- No incomplete certificate record is treated as a Core pass.
- No failed certificate record is treated as incomplete.
- No object-level gluing is inferred from certificate-record gluing.
- No problem-specific arithmetic, PDE, geometric, or categorical conclusion is asserted.
- No appendix or companion material enlarges the Core unless explicitly imported and reproved.
- No global exactness of T_τ^{bar} is assumed.
- No global strict filtered lift C_τ is assumed.
- No identity

$$C_{\tau, W}F = (C_\tau F)|_W$$

 is assumed.
- No tower comparison artifact is treated as automatic.

G.19. Provenance and status

The material in this appendix is built from:

- (i) the Minimal Manifest Axiom of Chapter 1;
- (ii) the repaired barcode-level collapse and window conventions of Chapters 2 and 5;
- (iii) the admissible representative discipline of Appendix B;
- (iv) the eligible realization discipline of Appendix C;
- (v) the monitored Type IV diagnostics of Chapter 4 and Appendix D;
- (vi) the audit pass/reject discipline of Chapter 6;
- (vii) the Core/non-claim separation of Chapter 7.

Its role is to define the abstract proof-object and certificate-record schema required for auditability.

Status labels for Appendix G. *Foundation definitions:* Definitions [6.3](#), [6.4](#), [6.5](#), [6.7](#), [6.8](#), [6.12](#), [6.13](#), [6.14](#), [6.17](#), [6.18](#), [6.19](#), [6.20](#), [6.23](#), [6.24](#), [6.25](#), [6.28](#), [6.29](#), [6.30](#), [6.33](#), [6.34](#), [6.35](#), [6.36](#), [6.39](#), [6.40](#), [6.41](#), [6.42](#), [6.45](#), [6.46](#), [6.47](#), [6.50](#), [6.51](#), [6.52](#), [6.55](#), [6.56](#), [6.60](#), [6.61](#), [6.64](#), [6.66](#), [6.68](#), [6.69](#), [6.72](#).

Foundation axioms/propositions/theorems: Axiom [6.1](#), Propositions [6.11](#), [6.16](#), [6.22](#), [6.27](#), [6.32](#), [6.38](#), [6.44](#), [6.49](#), [6.54](#), [6.57](#), [6.58](#), [6.62](#), [6.67](#), [6.71](#), [6.74](#), Theorem [6.75](#).

Operational cautions: Remarks [6.1](#), [6.2](#), [6.6](#), [6.9](#), [6.10](#), [6.15](#), [6.21](#), [6.26](#), [6.31](#), [6.37](#), [6.43](#), [6.48](#), [6.53](#), [6.59](#), [6.63](#), [6.65](#), [6.70](#), [6.73](#).

Explicit exclusions: Section [G.18](#).